

Christof Teuscher

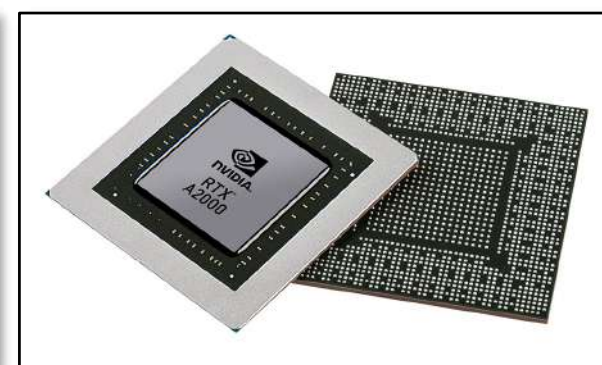
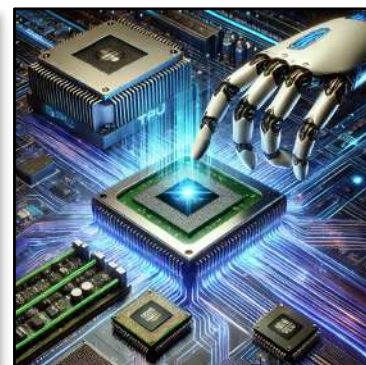
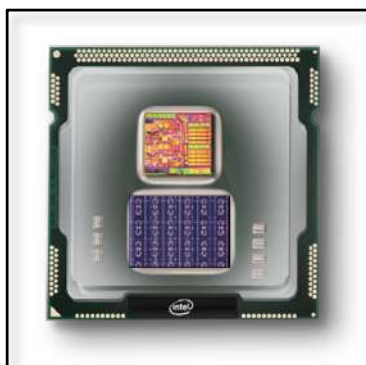
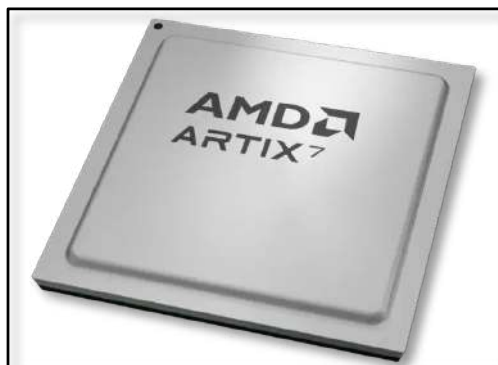
ECE 410/510: Hardware for AI and ML, Spring 2026

GPU Architecture, CUDA, and the Memory Wall

Portland State University
Department of Electrical and Computer Engineering (ECE)

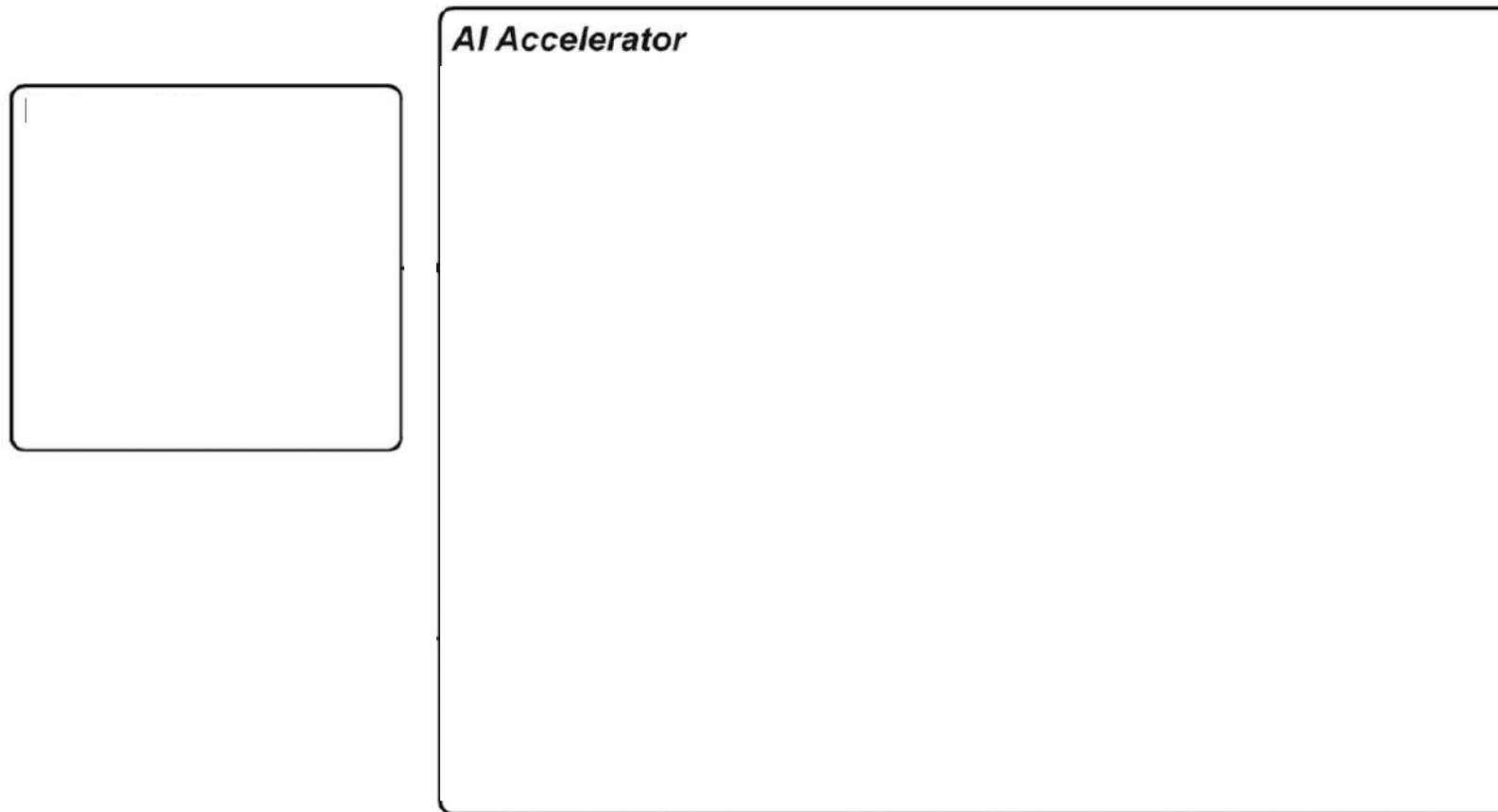
www.teuscher-lab.com

teuscher@pdx.edu



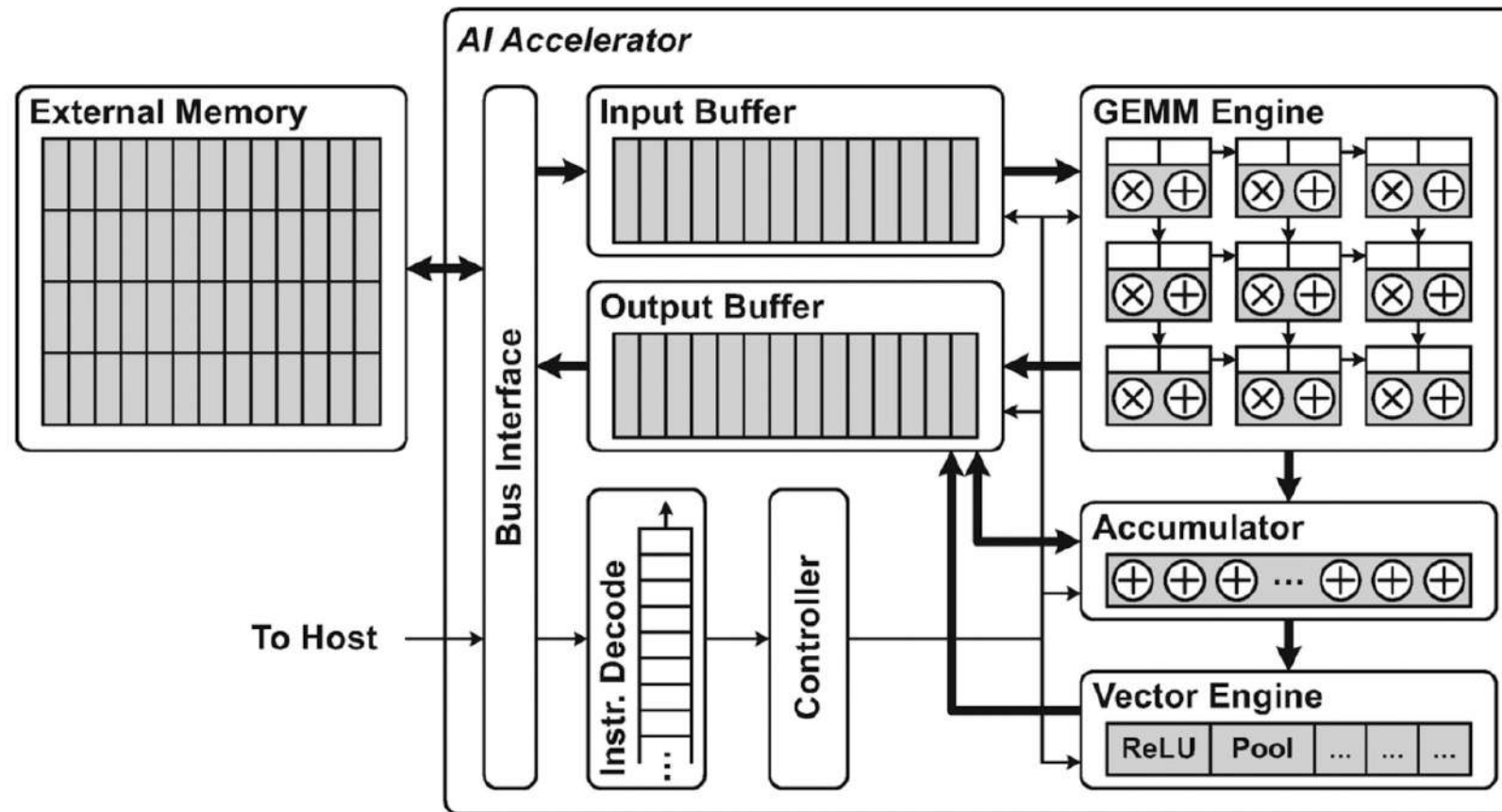


What are the components of a generic AI accelerator?



Where are potential bottlenecks?

General AI accelerator architecture



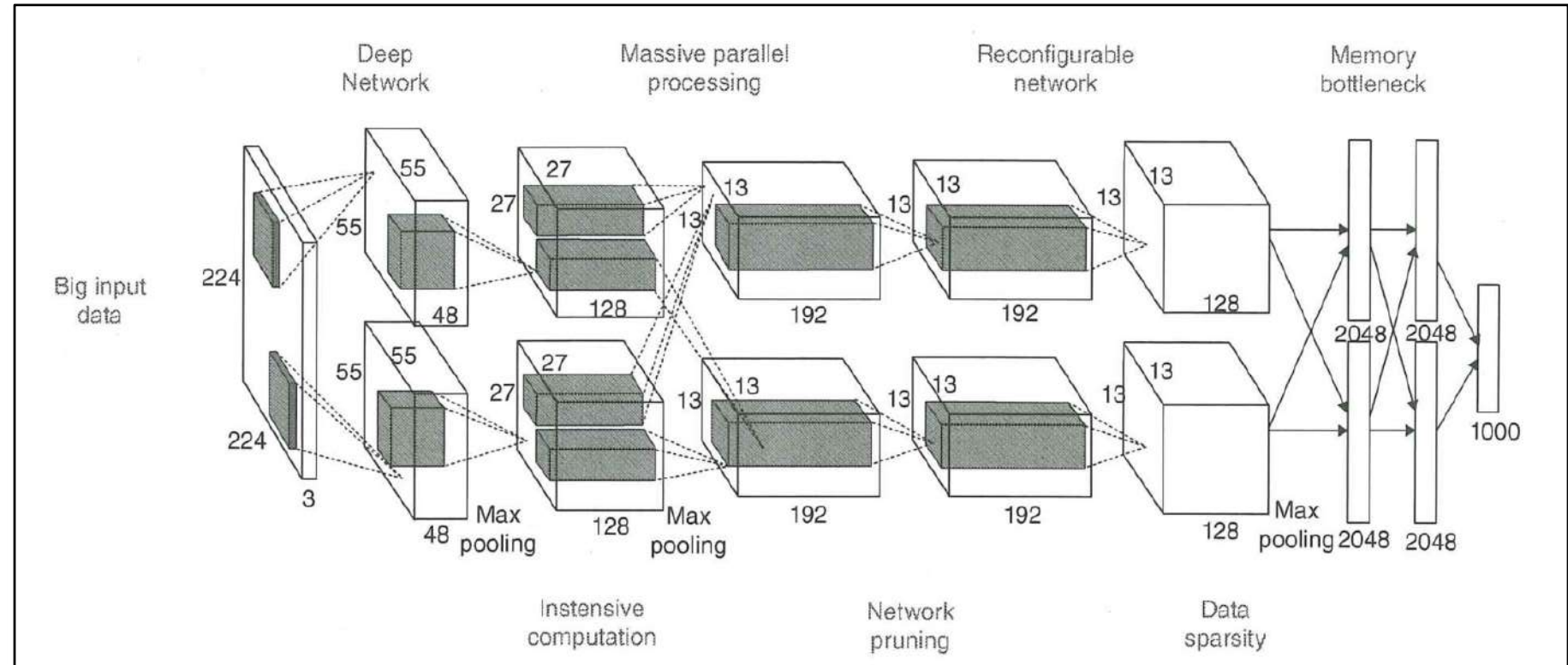
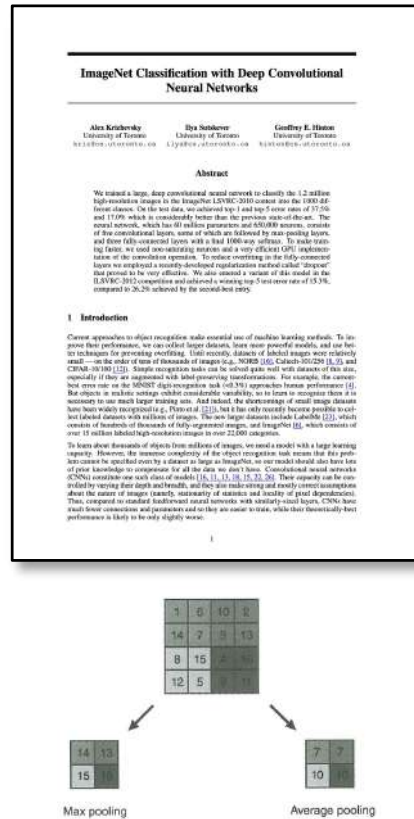
GEMM = General Matrix Multiply

Tensor cores!

A vector is a 1st-order tensor
A matrix is a 2nd-order tensor.

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

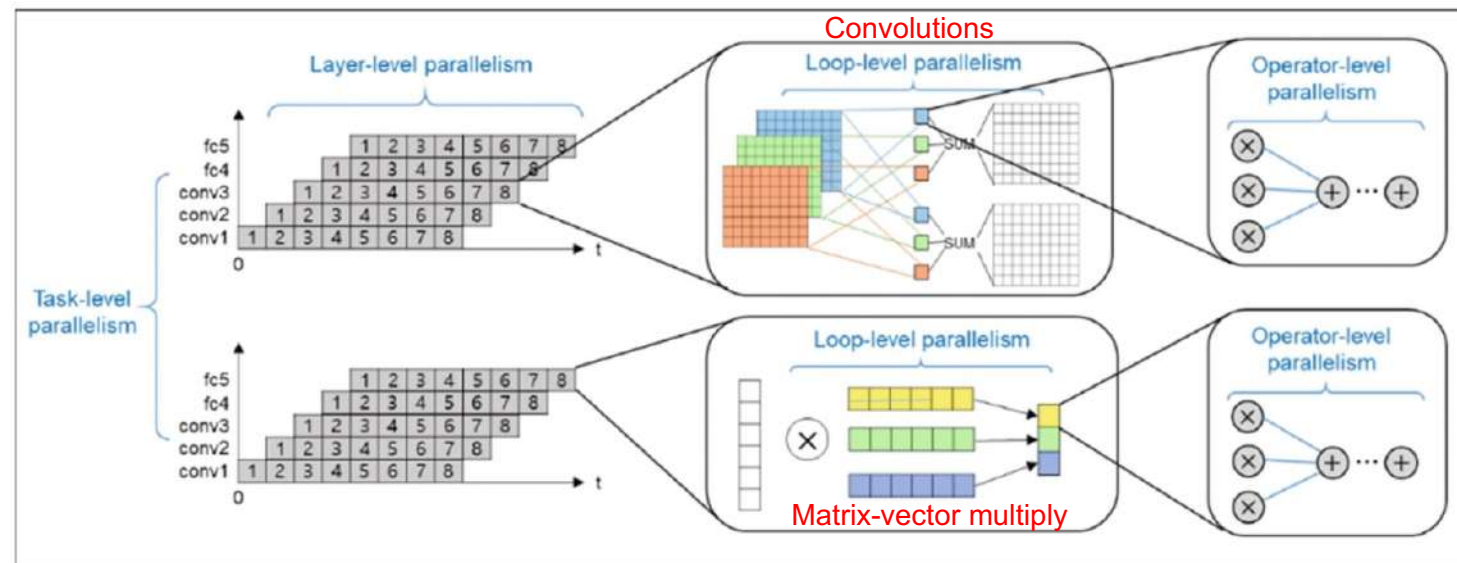
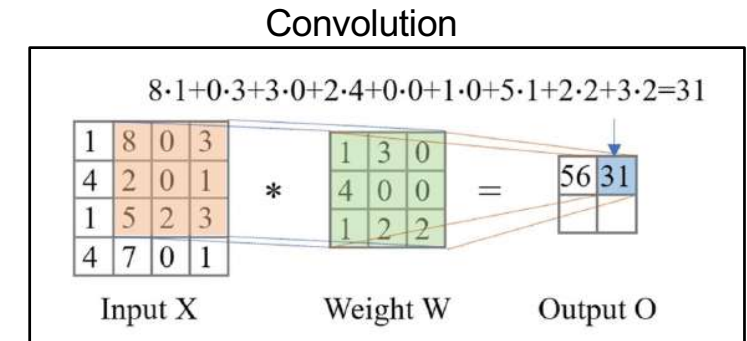
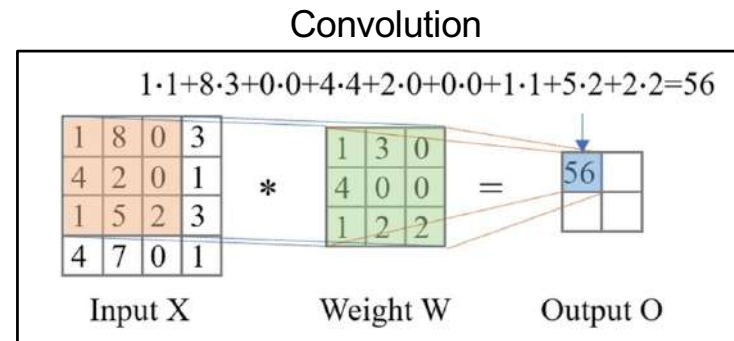
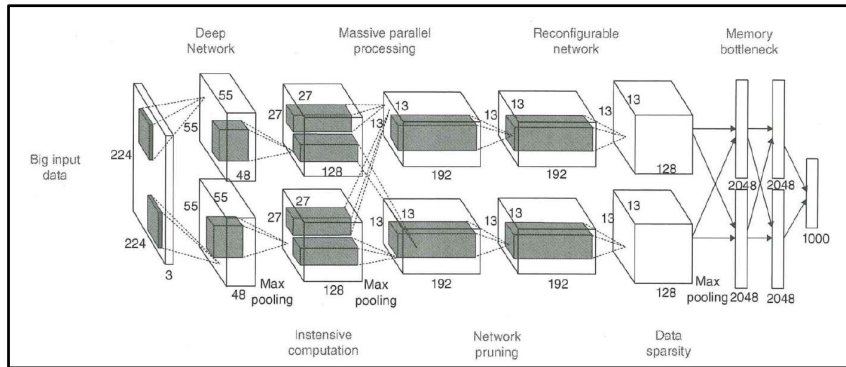
AlexNet: A classical deep convolutional neural network



AlexNet paper: https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

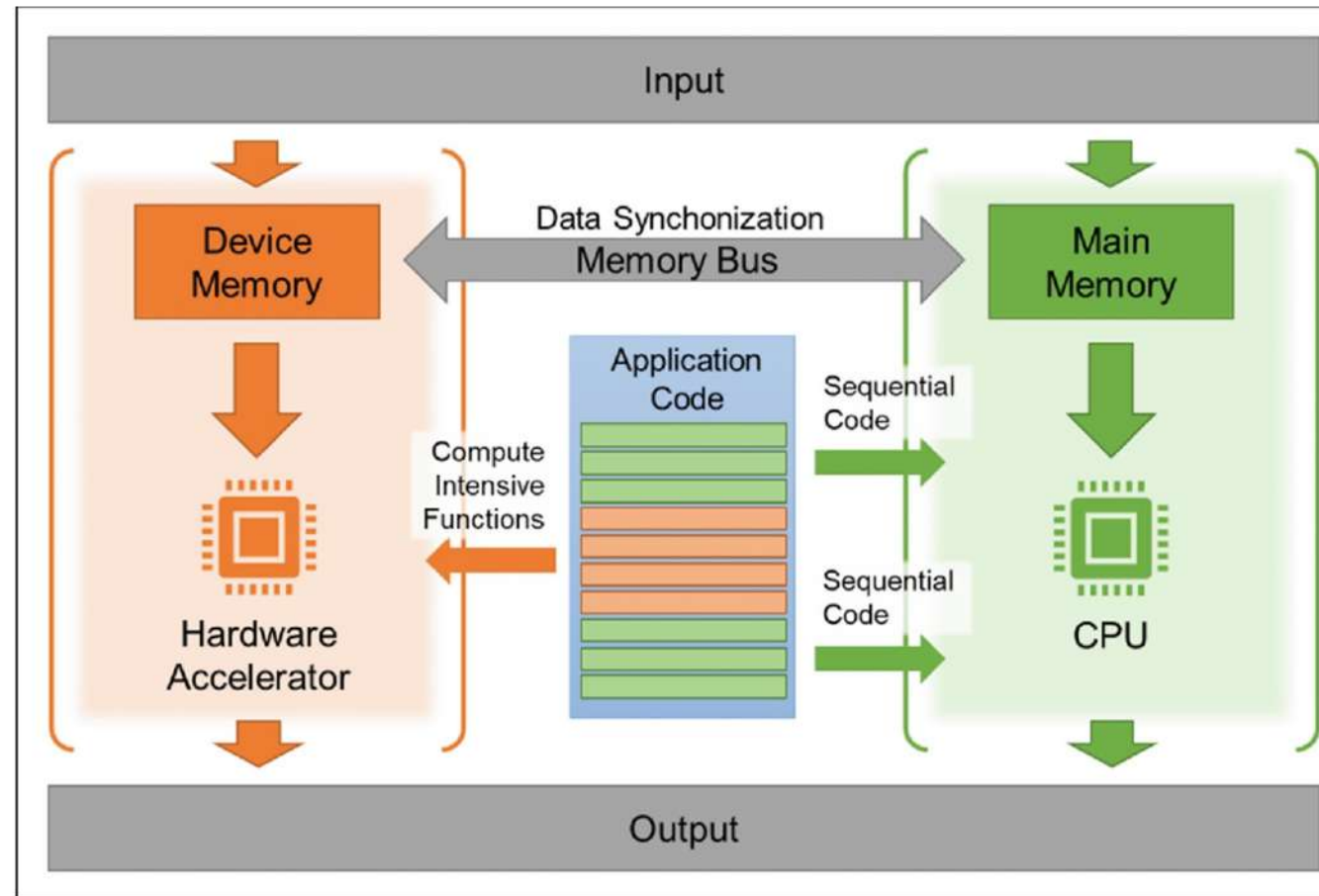
Liu & Law, Artificial Intelligence Hardware Design, 2021

What parallelism can be exploited?



- Max and avg. pooling are not convolutions.
- Similar operation, but convolutions have learnable parameters (weights).

General co-processing mechanism



Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

Hardware for AI/ML

Processor	Power Consumption	Strengths	Limitations
CPU	High	→ Flexible → General-purpose processing → Complex instructions and tasks → System management	→ Possible memory access bottlenecks → Few cores (4–16)
GPU	High	→ Parallel cores (~1000 s of cores) → High-performance AI processing	→ Power consumption → Large footprint
FPGA	Medium	→ Configurable logic gates → Flexible → In-field re-programmability	→ Programming complexity
ASIC	Low	→ Custom logic designed with libraries → Faster processing → Small footprint	→ Fixed function → Expensive custom design
Vision Processing Unit (VPU)	Ultra-low	→ Dedicated image and vision co-processor → Small footprint	→ Limited dataset and batch size → Limited network support
Tensor Processing Unit (TPU)	Low to medium	→ Specialized tool support → Optimized for Tensor-Flow	→ Proprietary design → Limited framework support



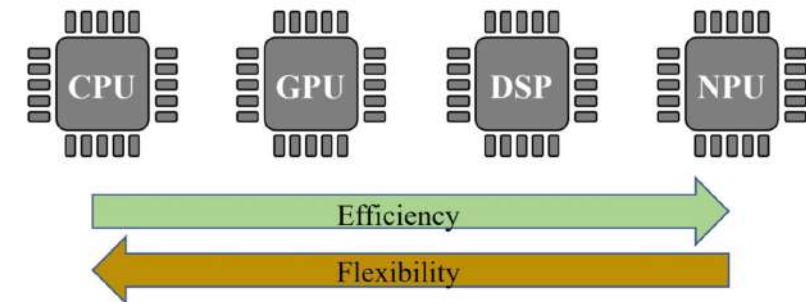
- Domain programmable
- Partially general-purpose
- Parallel workloads
- Power-hungry



- Hardware programmable
- Partially general-purpose
- Any workloads
- High performance
- Low power



- Hard wired
- Special-purpose
- Dedicated workloads
- High performance
- Low power





Christof Teuscher



teuscher@pdx.edu

www.teuscher-lab.com/teaching



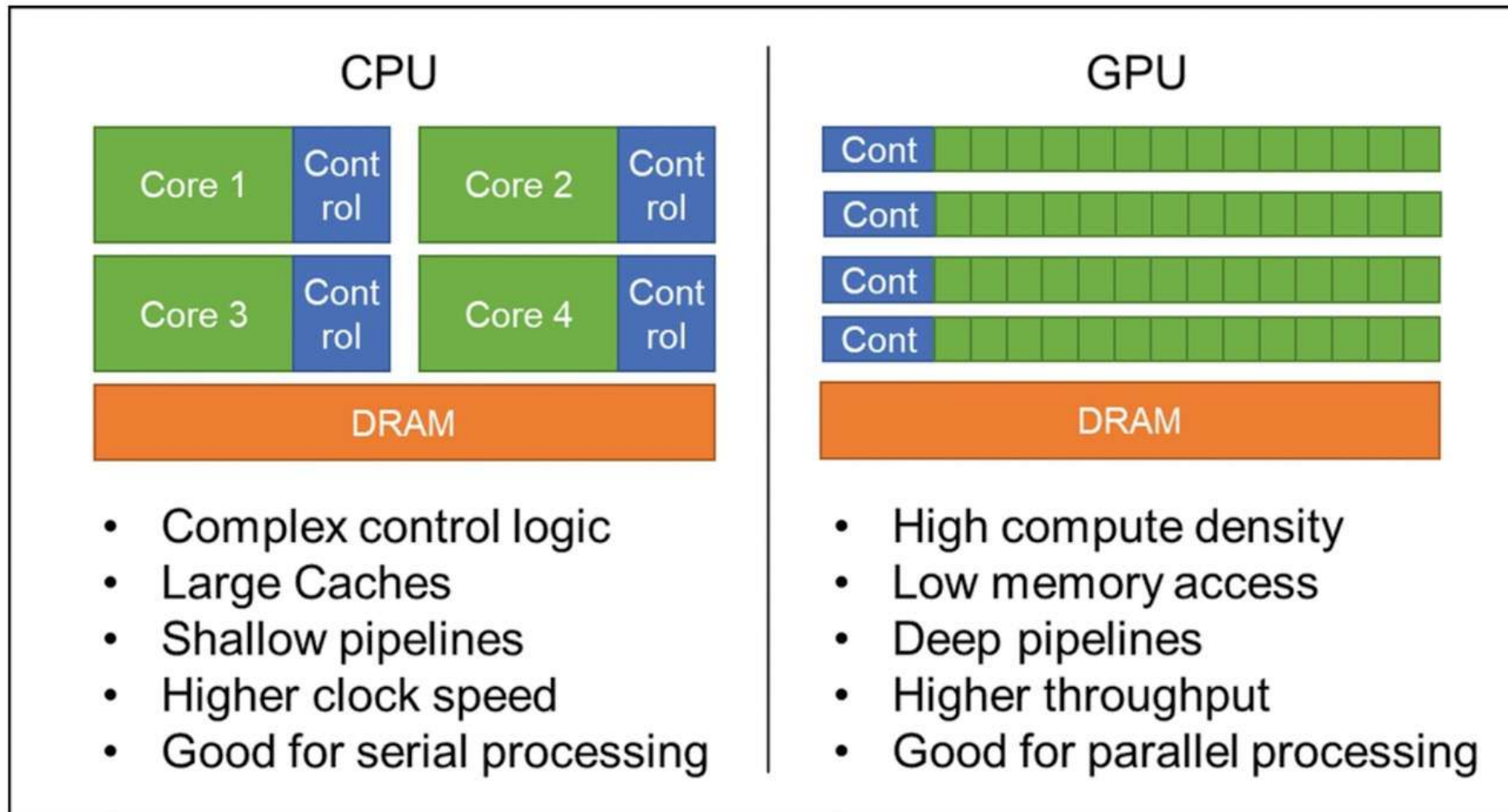
Graphics Processing Units (GPUs)

A bit of history

- Over several decades, GPUs evolved from a single core, fixed function hardware used for graphics solely, to a set of programmable parallel cores.
- The history of modern GPUs starts in 1995 with the introduction of the first 3D add-in cards.
- 1996: **3DFx's Voodoo** graphics card took over about 85% of the market. Cards that could only render 2D became obsolete very fast.
- 1999: **NVIDIA's GeForce 256**, the "world's first GPU."



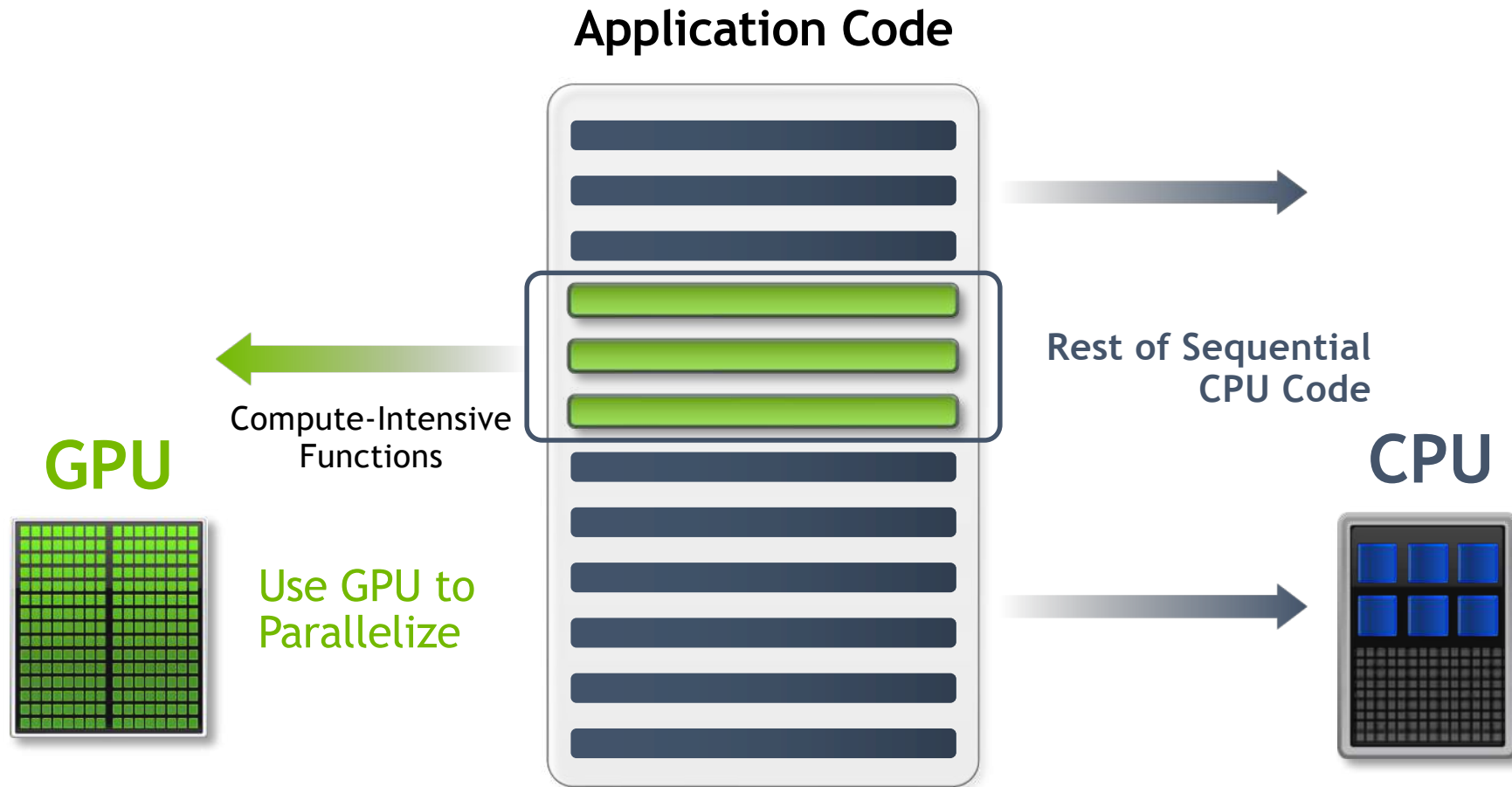
Multi-core vs GPU



SIMT (Single Instruction, Multiple Threads) is a parallel processing model used in GPUs where multiple threads execute the same instruction simultaneously, but on different data.

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

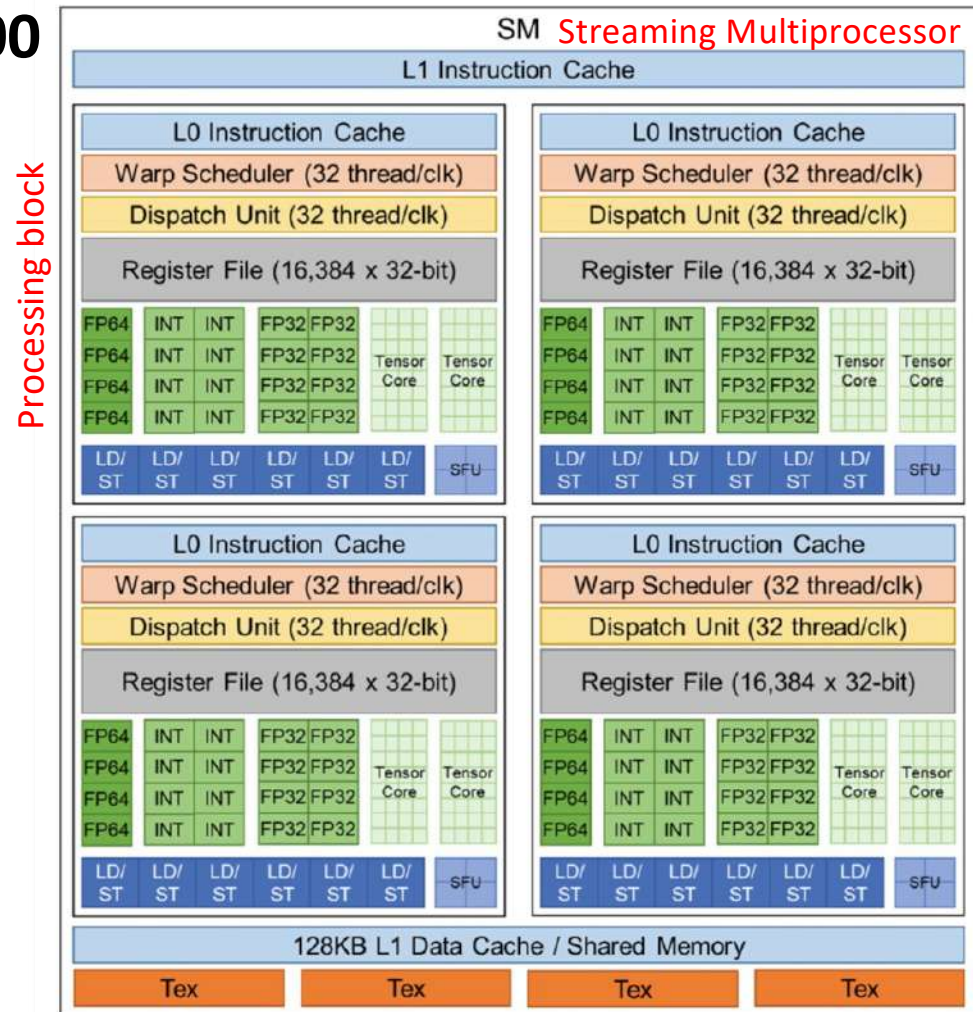
Small Changes, Big Speed-up



NVIDIA

NVIDIA Volta Microarchitecture (2013-2017)

V100

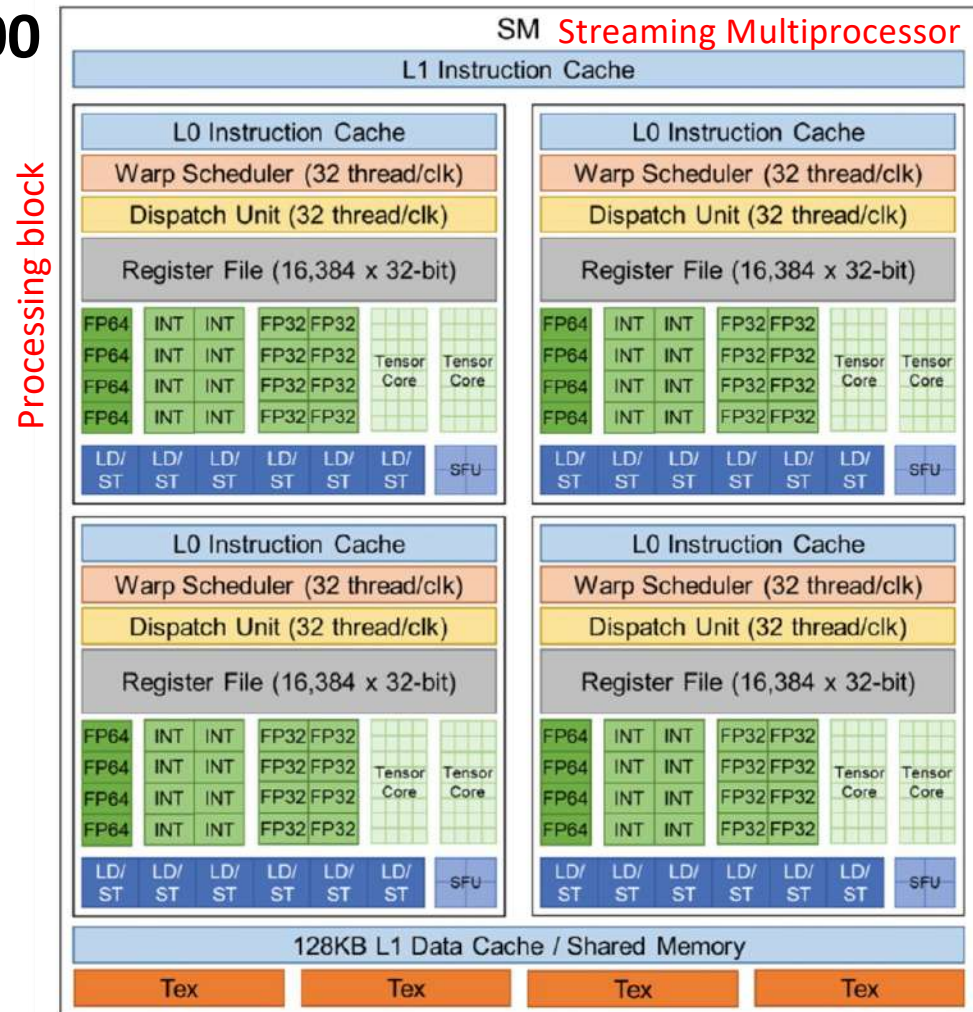


Spec	P100	V100	A100	H100	H200	B100	B200	B300
Architecture	Pascal	Volta	Ampere	Hopper	Hopper	Blackwell	Blackwell	Blackwell Ultra
Release year	2016	2017	2020	2022	2024	2025	2025	2025
Process node	16nm	12nm	7nm	4nm	4nm	4NP	4NP	4NP
Die config	Single	Single	Single	Single	Single	Dual	Dual	Dual
FP32 TFLOPS	9.3	14	19.5	67	67	45	75	~90
FP16 TFLOPS	18.7	28	77.6	989	1,979	1,750	2,250	~3,500
BF16 TFLOPS	—	—	77.6	989	1,979	1,750	2,250	~3,500
FP8 TFLOPS	—	—	—	1,979	3,958	3,500	4,500	~7,000
FP4 TOPS	—	—	—	—	—	—	9,000	15,000
INT8 TOPS	—	—	624	1,979	3,958	7,000	9,000	—
Tensor cores	—	1st gen	3rd gen	4th gen	4th gen	5th gen	5th gen	5th gen
HBM gen	HBM2	HBM2	HBM2e	HBM3	HBM3e	HBM3e	HBM3e	HBM3e
HBM capacity	16 GB	16/32 GB	40/80 GB	80 GB	141 GB	192 GB	192 GB	288 GB
Memory bandwidth	732 GB/s	900 GB/s	2 TB/s	3.35 TB/s	4.8 TB/s	8 TB/s	8 TB/s	8 TB/s
NVLink bandwidth	160 GB/s	300 GB/s	600 GB/s	900 GB/s	900 GB/s	1.8 TB/s	1.8 TB/s	1.8 TB/s
TDP	300 W	300 W	400 W	700 W	700 W	700 W	1,000 W	1,400 W

Why not 4x more throughput?

NVIDIA Volta Microarchitecture (2013-2017)

V100



Spec	P100	V100	A100	H100	H200	B100	B200	B300
Architecture	Pascal	Volta	Ampere	Hopper	Hopper	Blackwell	Blackwell	Blackwell Ultra
Release year	2016	2017	2020	2022	2024	2025	2025	2025
Process node	16nm	12nm	7nm	4nm	4nm	4NP	4NP	4NP
Die config	Single	Single	Single	Single	Single	Dual	Dual	Dual
FP32 TFLOPS	9.3	14	19.5	67	67	45	75	~90
FP16 TFLOPS	18.7	28	77.6	989	1,979	1,750	2,250	~3,500
BF16 TFLOPS	—	—	77.6	989	1,979	1,750	2,250	~3,500
FP8 TFLOPS	—	—	—	1,979	3,958	3,500	4,500	~7,000
FP4 TOPS	—	—	—	—	—	—	9,000	15,000
INT8 TOPS	—	—	624	1,979	3,958	7,000	9,000	—
Tensor cores	—	1st gen	3rd gen	4th gen	4th gen	5th gen	5th gen	5th gen
HBM gen	HBM2	HBM2	HBM2e	HBM3	HBM3e	HBM3e	HBM3e	HBM3e
HBM capacity	16 GB	16/32 GB	40/80 GB	80 GB	141 GB	192 GB	192 GB	288 GB
Memory bandwidth	732 GB/s	900 GB/s	2 TB/s	3.35 TB/s	4.8 TB/s	8 TB/s	8 TB/s	8 TB/s
NVLink bandwidth	160 GB/s	300 GB/s	600 GB/s	900 GB/s	900 GB/s	1.8 TB/s	1.8 TB/s	1.8 TB/s
TDP	300 W	300 W	400 W	700 W	700 W	700 W	1,000 W	1,400 W

CUDA core TP
Different execution units.
See diagram!
Tensor core TP

H100 SM architecture (Hopper, 2022)

□ **Key H100 SXM5 specifications**

- 132 SMs; 16,896 CUDA cores; 528 4th-gen Tensor Cores
- 80 GB HBM3; 3.35 TB/s memory bandwidth
- 228 KB shared memory per SM (L1/shared configurable)
- Peak FP32: 67 TFLOP/s; Peak FP8 (Tensor): 3,958 TFLOP/s

□ **SM internal hierarchy**

- Each SM has 4 processing blocks; each block has 1 warp scheduler + 32 FP32 cores
- Tensor Cores sit alongside the FP32 units and execute 4x4 MMA in one clock
- Register file: 65,536 x 32-bit registers per SM; shared across all active warps

□ **What's the ridge point?**

H100 SM architecture (Hopper, 2022)

□ Key H100 SXM5 specifications

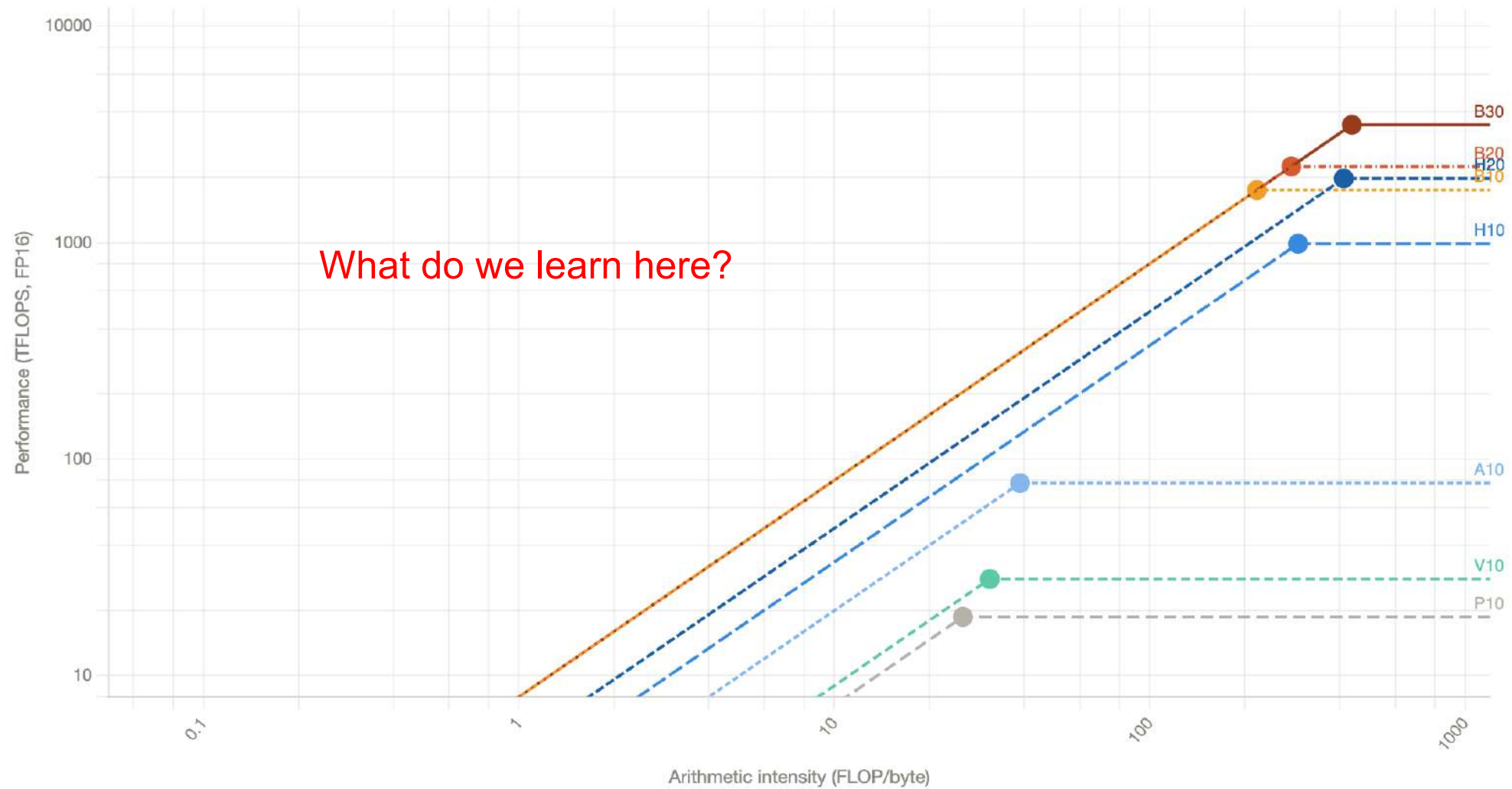
- 132 SMs; 16,896 CUDA cores; 528 4th-gen Tensor Cores
- 80 GB HBM3; **3.35 TB/s** memory bandwidth
- 228 KB shared memory per SM (L1/shared configurable)
- Peak FP32: **67 TFLOP/s**; Peak FP8 (Tensor): 3,958 TFLOP/s

□ SM internal hierarchy

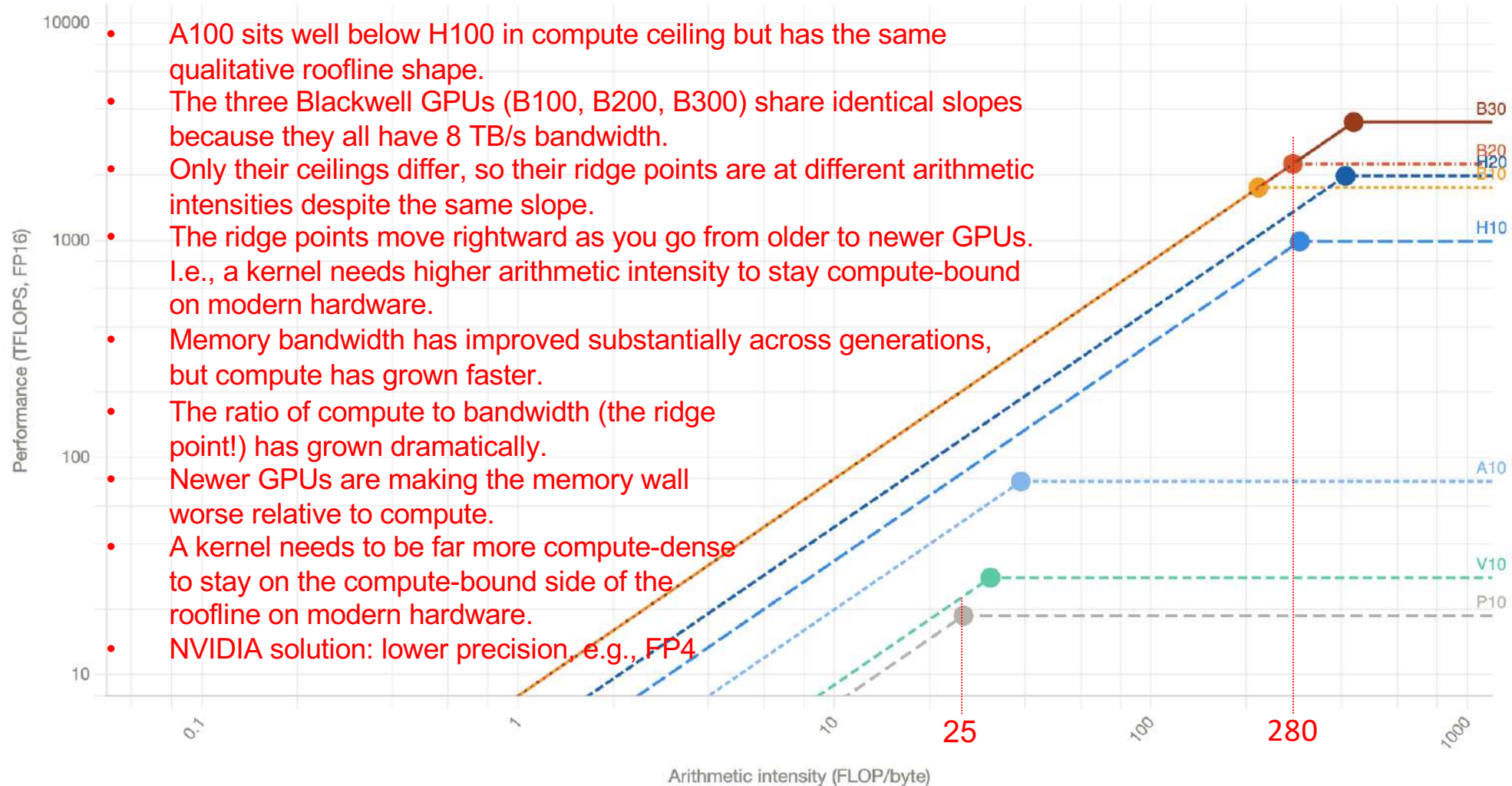
- Each SM has 4 processing blocks; each block has 1 warp scheduler + 32 FP32 cores
- Tensor Cores sit alongside the FP32 units and execute 4x4 MMA in one clock.
- Register file: 65,536 x 32-bit registers per SM; shared across all active warps

□ **Ridge point = Peak FLOP/s / Peak bandwidth = 67 / 3.35 $\approx$ 20 FLOP/byte**

Rooflines (FP16)

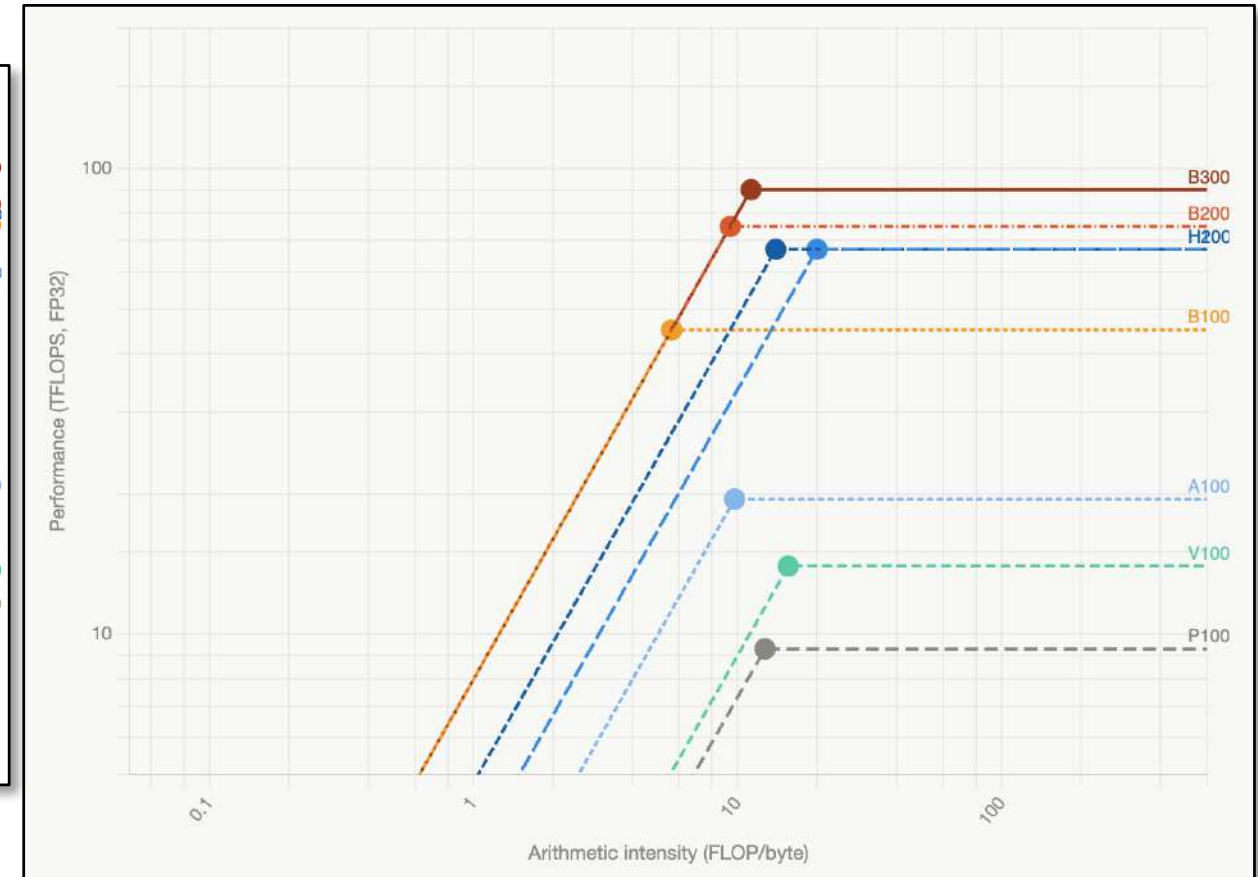
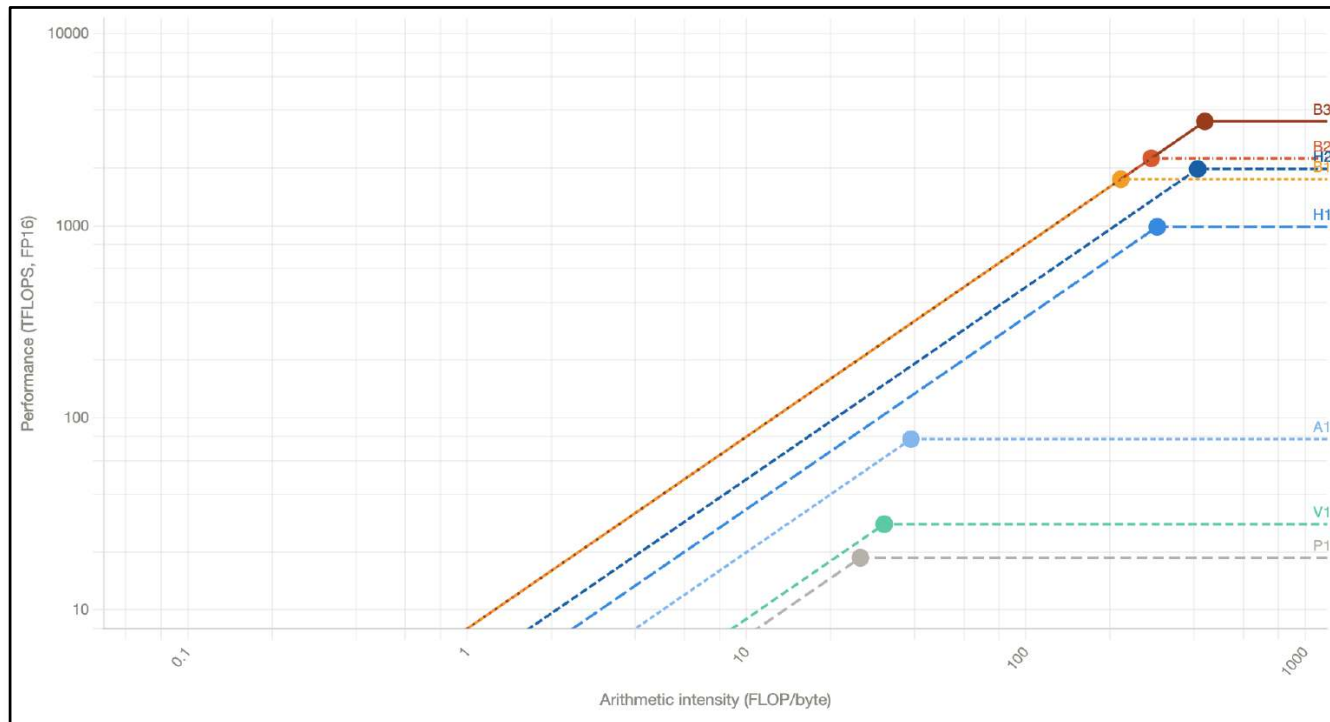


Rooflines (FP16)



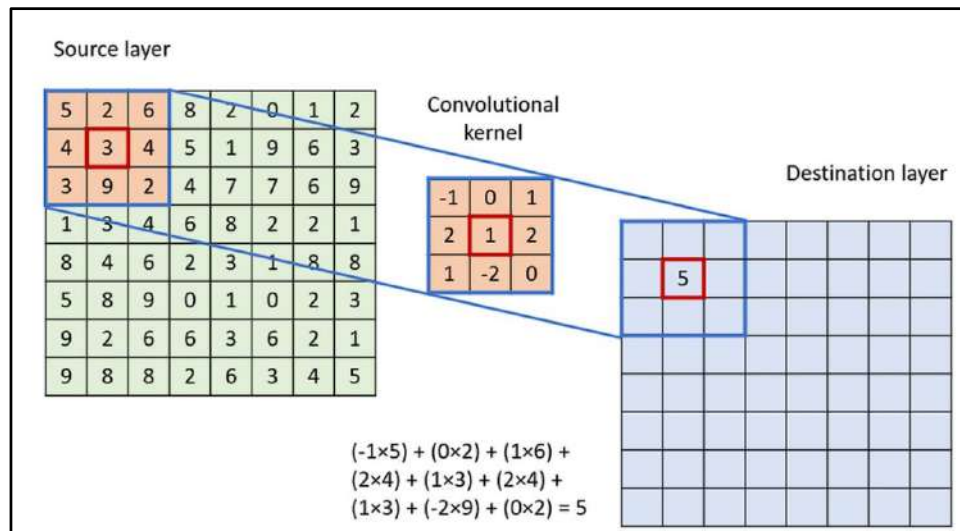
Rooflines (FP32)

What changed compared to FP16? Why?



Tensor cores (1)

- Each tensor core executes MMA (matrix multiplication and accumulation, $D = A \times B + C$) on a 4×4 floating-point matrix.
- They are built to enhance deep learning by boosting matrix arithmetic.
- Low-precision operations often supported.



Convolution

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

MMA

Tensor cores (2)

Tensor Cores operate on FP16 input data with FP32 accumulation. The FP16 multiply results in a full precision product that is then accumulated using FP32 addition with the other intermediate products for a 4x4x4 matrix multiply (see Figure 9). In practice, Tensor Cores are used to perform much larger 2D or higher dimensional matrix operations, built up from these smaller elements.

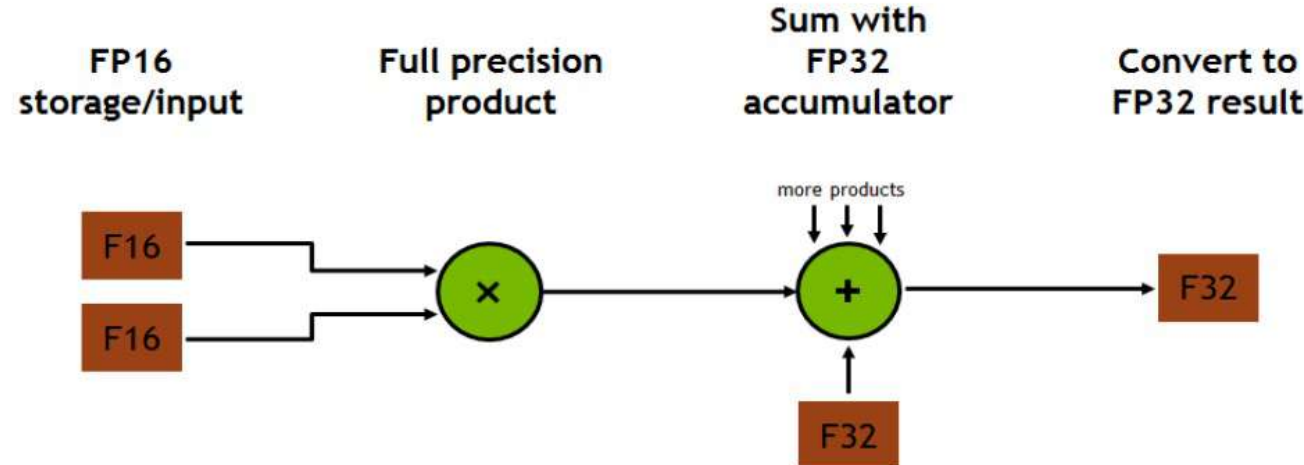
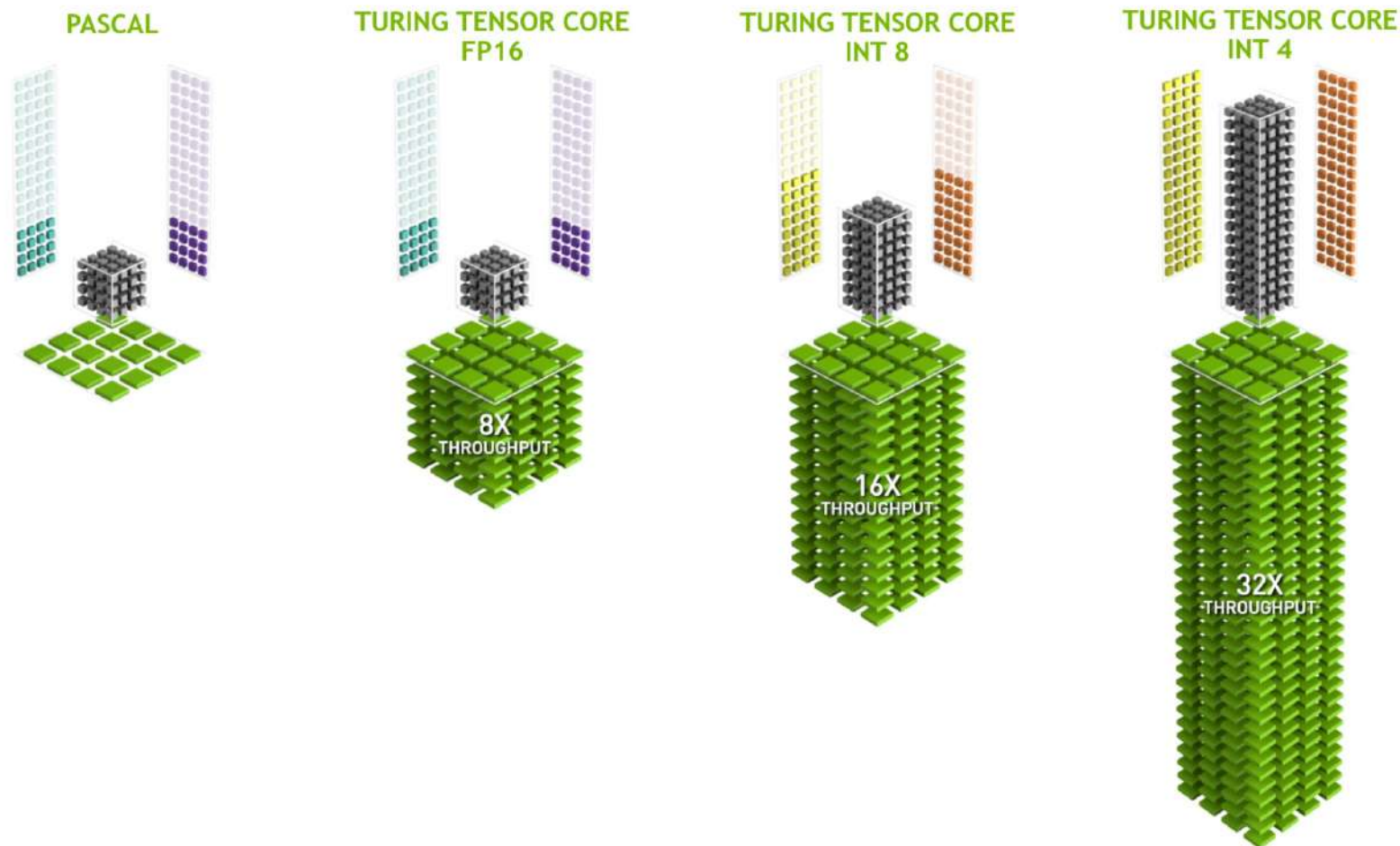


Figure 9. Mixed Precision Multiply and Accumulate in Tensor Core

Tensor cores (3)



New Turing Tensor Cores Provide Multi-Precision for AI Inference.

Streaming vs tensor cores

SM Cores (Streaming Multiprocessors)

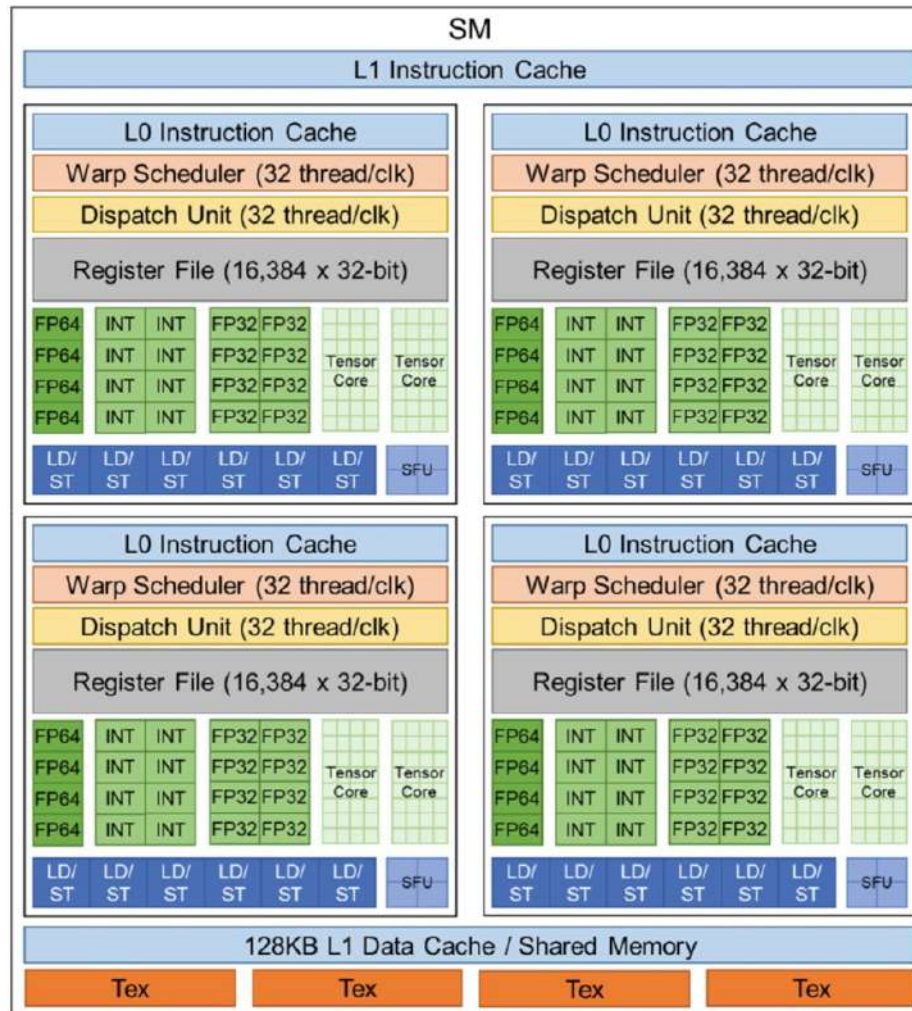
- These are the traditional GPU computing units
- Handle general-purpose parallel computing tasks
- Process standard graphics rendering workloads
- Execute regular FP32 (single precision) and INT32 operations
- Responsible for most traditional GPU workloads

Tensor Cores

- Specialized hardware accelerators designed specifically for matrix operations
- Optimized for deep learning and AI workloads
- Dramatically accelerate matrix multiply-accumulate operations (critical for neural networks)
- Can perform mixed-precision calculations (like FP16 and FP32)
- Provide order-of-magnitude speedups for deep learning training and inference
- Introduced with NVIDIA's Volta architecture and improved in subsequent generations

Spec	P100	V100	A100	H100	H200	B200
Architecture	Pascal	Volta	Ampere	Hopper	Hopper	Blackwell
SM count	56	80	108	132	132	148
CUDA cores/SM	64	64	128	128	128	128
Total CUDA cores	3,584	5,120	6,912	16,896	16,896	16,896
Tensor core gen	—	1st	3rd	4th	4th	5th
Tensor cores/SM	—	8	4	4	4	4
Total tensor cores	—	640	432	528	528	528
FLOPs/TC/cycle (FP16)	—	256	512	1,024	1,024	~2,048
Shared mem/SM	64 KB	96 KB	164 KB	228 KB	228 KB	228 KB
Register file/SM	256 KB	256 KB	256 KB	256 KB	256 KB	256 KB
Tensor mem (TMEM)/SM	—	—	—	—	—	256 KB
Warps/SM (max)	64	64	64	64	64	64
Tensor core precision	—	FP16, INT8	FP16, BF16, TF32, FP64, INT8	FP8, FP16, BF16, TF32, FP64, INT8	same as H100	FP4, FP6, FP8, FP16, BF16, TF32

V100



A100

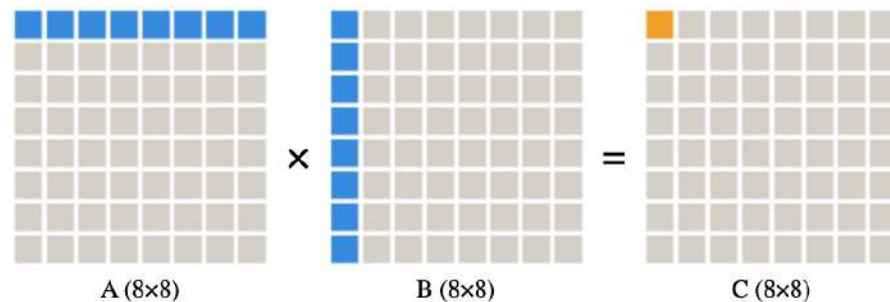


The memory wall

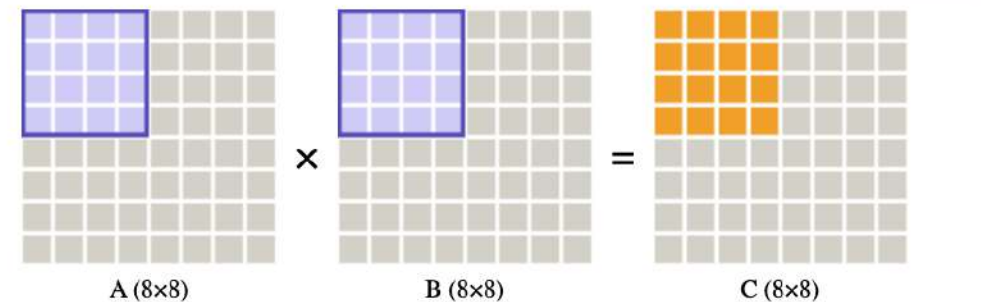
- **The memory wall: compute has outpaced bandwidth for 30 years**
 - H100: 67 TFLOP/s (FP32) but only 3.35 TB/s HBM3 bandwidth
 - Arithmetic intensity needed to stay compute-bound: $67 / 3.35 \approx 20$ FLOP/byte
 - Most real workloads fall below this, i.e., memory is the bottleneck
- **DRAM vs SRAM:**
 - SRAM (on-chip shared memory): ~1 TB/s, but only 228 KB per SM on H100.
 - DRAM (HBM3): 3.35 TB/s aggregate, but 80 GB, i.e., 3 orders of magnitude more.
 - Off-chip DRAM latency: ~100× slower than on-chip SRAM.
- **Energy cost of data movement (Horowitz, 2014):**
 - FP32 multiply: ~3.7 pJ
 - DRAM read (64-bit): ~640 pJ, i.e., roughly 170× more expensive.
 - Minimising DRAM traffic is therefore both a performance and energy objective.

Case study: Naïve vs tiled GEMM

Naïve GEMM



Tiled GEMM



- In **naïve GEMM**, computing each output element $C[i,j]$ requires loading the entire row i of A and column j of B from DRAM. Those same elements are loaded again for every other output element in the same row or column. No data is reused, i.e., total DRAM traffic scales as N^3 .
- In **tiled GEMM**, a $T \times T$ tile of A and a $T \times T$ tile of B are loaded once into shared memory (shown in purple). Every thread in the tile then computes against that shared data without going back to DRAM. **Each tile load is reused T times before the next tile is fetched.** Total DRAM traffic drops by a factor of T , which is exactly the arithmetic intensity improvement. I.e., you do the same number of MACs but move T times fewer bytes.
- **For your chiplet designs: this is the on-chip memory argument made concrete. Sizing the scratchpad or local SRAM to hold at least one full tile is what makes the difference between a memory-bound and a compute-bound design on the roofline.**
- Good explanation: <https://www.youtube.com/watch?v=Q3GgbfGTnVc>

Naive GEMM: DRAM traffic analysis

- **Naive matrix multiply ($M=N=K=64$, FP32):**
 - Each output element $C[i,j]$ requires: 1 row of A (64 floats) + 1 col of B (64 floats)
 - No reuse: every element loaded fresh from DRAM for each output
- **DRAM reads:**
 - A: $M \times K \times N$ reads = $64 \times 64 \times 64 = 262,144$ floats = 1,048,576 bytes (1 MB)
 - B: $N \times K \times M$ reads = $64 \times 64 \times 64 = 262,144$ floats = 1,048,576 bytes (1 MB)
 - Total DRAM traffic: 2 MB reads to produce 16 KB of output ($64 \times 64 \times 4$ bytes)
- **Arithmetic intensity:**
 - FLOPs: $2 \times 64 \times 64 \times 64 = 524,288$
 - Bytes: 2,097,152 (reads) + 16,384 (writes, negligible) ≈ 2.1 MB
 - AI = $524,288 / 2,097,152 \approx 0.25$ FLOP/byte, i.e., deep in memory-bound regime.

Tiled GEMM: DRAM traffic analysis

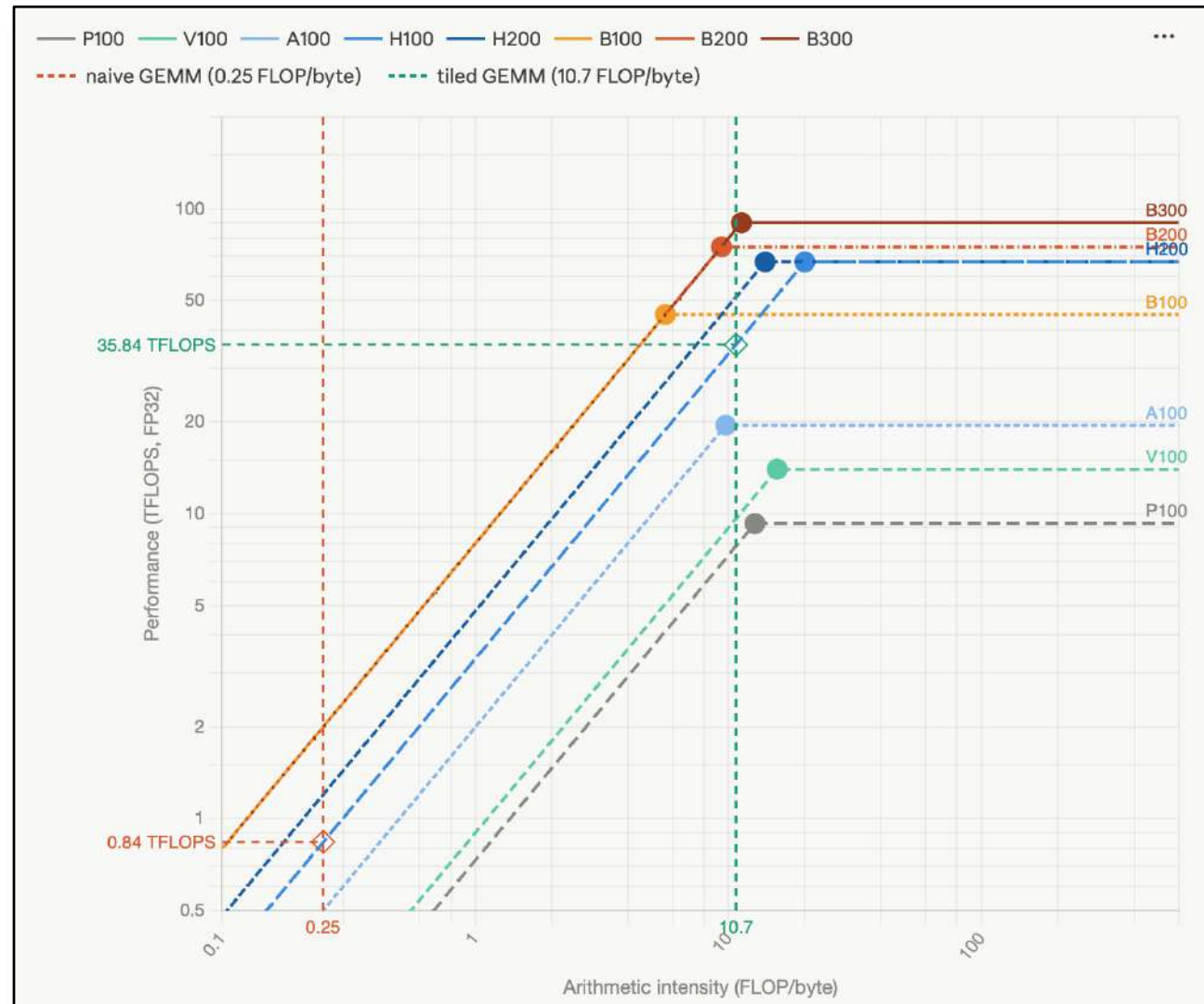
- **Tiled matrix multiply (tile size $T=16$, $M=N=K=64$, FP32):**
 - Load a $T \times T$ tile of A and $T \times T$ tile of B into shared memory
 - Each tile element is reused T times for dot products within the tile
- **DRAM reads:**
 - Every element of A and B is read from DRAM exactly once.
 - A: $M \times K$ floats loaded once = $64 \times 64 = 4,096$ floats = 16,384 bytes (16 KB)
 - B: $K \times N$ floats loaded once = $64 \times 64 = 4,096$ floats = 16,384 bytes (16 KB)
 - Total DRAM traffic: 32 KB, i.e., **a 64× reduction over naive** (= tile size $T=16^2/4$)
- **Arithmetic intensity:**
 - FLOPs unchanged: 524,288
 - Bytes: 32,768 (reads) + 16,384 (writes) $\approx$ 49 KB
 - $AI = 524,288 / 49,152 \approx 10.7$ FLOP/byte, i.e., approaching ridge point on H100.
- **Larger tiles $\rightarrow$ higher reuse $\rightarrow$ closer to compute-bound. Limit: shared memory size.**

Roofline placement: naive vs. tiled GEMM

- **H100 roofline reference points:**
 - Peak FP32 compute: 67 TFLOP/s
 - Peak High Bandwidth Memory 3 (HBM3) : 3.35 TB/s
 - Ridge point: $67\text{e}12 / 3.35\text{e}12 \approx 20$ FLOP/byte
- **Naive 64×64 GEMM: AI ≈ 0.25 FLOP/byte**
 - 80× below the ridge point, i.e., entirely memory-bound
 - Attainable performance: $0.25 \times 3.35\text{e}12 \approx 0.84$ TFLOP/s (1.25% of peak)
- **Tiled 64×64 GEMM (T=16): AI ≈ 10.7 FLOP/byte**
 - Still below ridge point but 43× better than naive
 - Attainable performance: $10.7 \times 3.35\text{e}12 \approx 35.8$ TFLOP/s (53% of peak)
- **Lesson: tiling moves the operating point rightward on the roofline.**
 - This is what you will measure in Nsight on Wednesday.

Roofline placement: naive vs. tiled GEMM

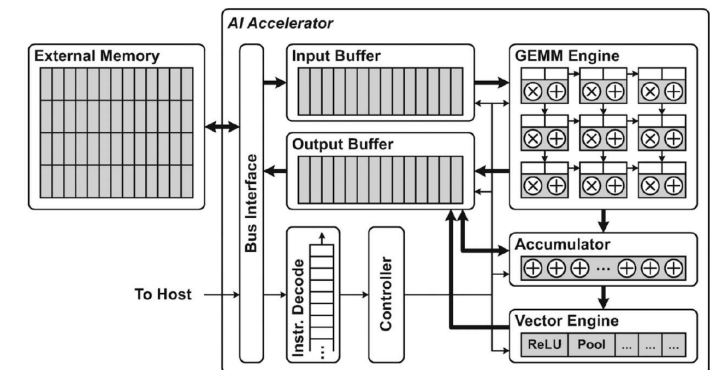
Lesson: tiling moves the operating point rightward on the roofline.



H100

Project: Accelerator datapath template

- **Use the general AI accelerator block diagram as a project template:**
 - Host interface: AXI4-Lite (control) + AXI4-Stream (data) — your M1 choice.
 - Input buffer: on-chip SRAM sized to one tile of your dominant kernel
 - Compute engine: the kernel-specific datapath (MAC array, dot-product unit, etc.)
 - Accumulator: partial sums before writing back to output buffer
 - Output buffer: holds results before DMA to host
 - Controller: FSM driving tile iteration and interface transactions
- **Design questions to answer for your block diagram:**
 - What is your dominant kernel? (from profiling)
 - What tile size fits in your on-chip SRAM budget?
 - What precision does your compute engine use? (FP32 / INT8 / BF16)
 - Is your kernel compute-bound or memory-bound at that tile size?

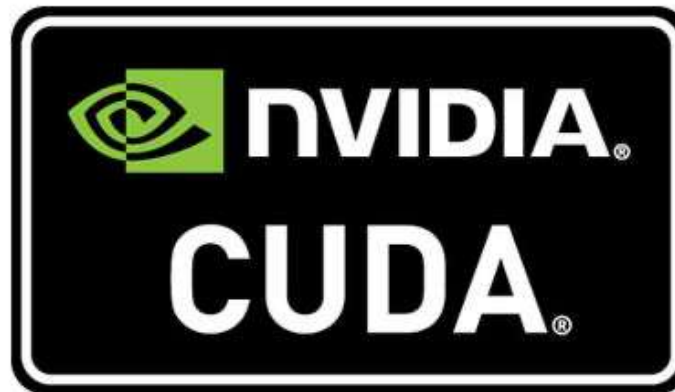


Project: HW/SW partitioning decision criteria

- **Three criteria for deciding what to accelerate in hardware:**
 - 1. Does the kernel dominate total runtime? (From profiling — Amdahl's law applies).
 - 2. Is the kernel compute-bound or memory-bound at the target tile size?
 - 3. Does the kernel have regular enough data access patterns to map to a fixed datapath?
- **Compute-bound → good candidate: parallelize the arithmetic**
 - Dense GEMM, convolution, dot products over large vectors
- **Memory-bound → harder: need to restructure data access first**
 - Sparse ops, embedding lookups, graph message passing
 - Must show tiling or fusion closes the gap before committing to HW
- **Irregular access patterns → reconsider scope**
 - If data-dependent branching or pointer chasing, a fixed datapath will stall
 - This is a common over-scoping failure mode in the project

CUDA

- **CUDA** (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA.
- It enables developers to use NVIDIA graphics processing units (GPUs) for general-purpose computing, a practice known as **GPGPU** (General-Purpose computing on Graphics Processing Units).



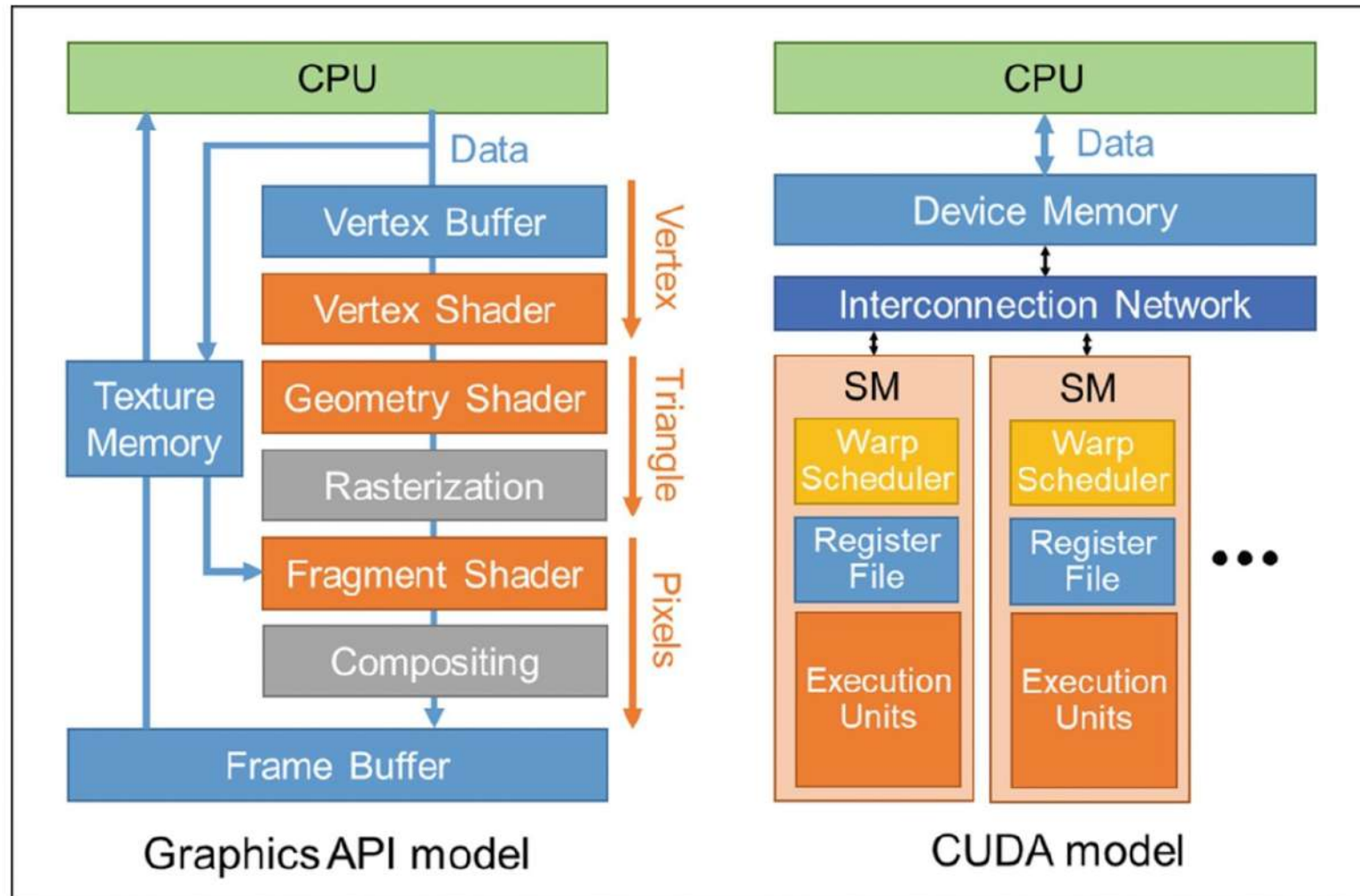
- Good resource: <https://docs.nvidia.com/cuda/cuda-c-programming-guide>

CUDA model

Good resource: <https://docs.nvidia.com/cuda/cuda-c-programming-guide>



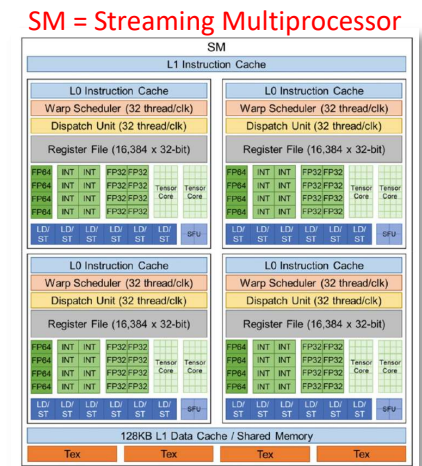
CUDA (**Compute Unified Device Architecture**) is a parallel computing platform and programming model created by NVIDIA that allows software developers to use NVIDIA GPUs for **general-purpose processing**, known as **GPGPU**



Early GPGPU was implemented by formulating data and operations in graphics primitives and translated to pass through a fixed graphics pipeline.

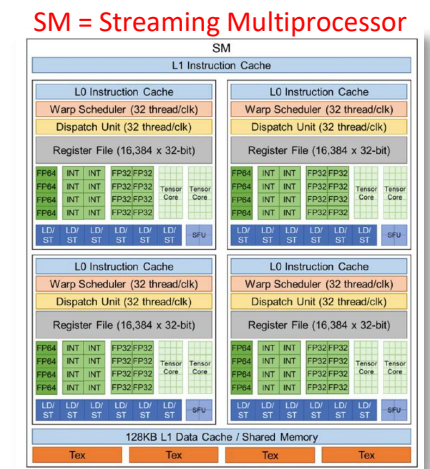
Warp

- A "warp" is a group of threads that execute instructions simultaneously on a GPU (Graphics Processing Unit).
- The warp scheduler is a component of the GPU architecture that manages the execution of these thread groups.
- **A warp typically consists of 32 threads in NVIDIA GPUs** (AMD calls similar groupings "wavefronts"). These threads execute the same instruction at the same time in a **SIMT** (Single Instruction, Multiple Thread) architecture.
- The warp scheduler is responsible for:
 - Deciding which warps are ready to execute.
 - Allocating compute resources to warps.
 - Managing warp context switching when threads need to wait for memory operations.
 - Hiding memory latency by switching between warps.
- **Warp divergence (key failure mode)**
 - ☐ If threads in a warp take different branches (if/else), the GPU serializes both paths.
 - ☐ Threads on the idle path are masked off. Throughput drops by up to 2x per branch level.
 - ☐ Avoid control flow that depends on thread ID. Keep all 32 threads on the same path.



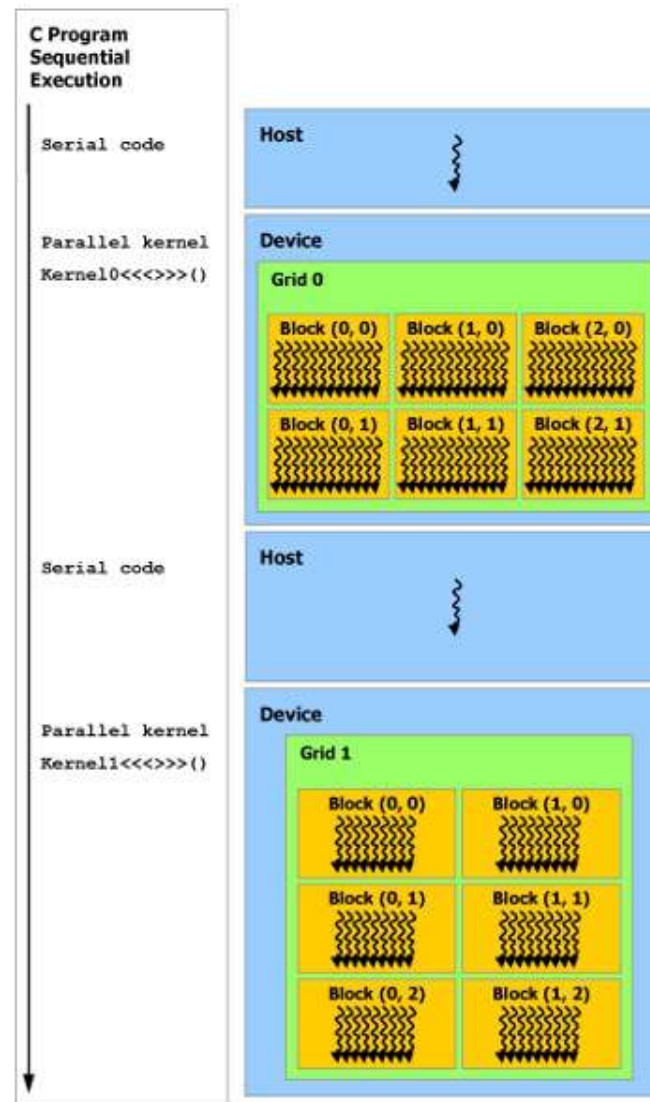
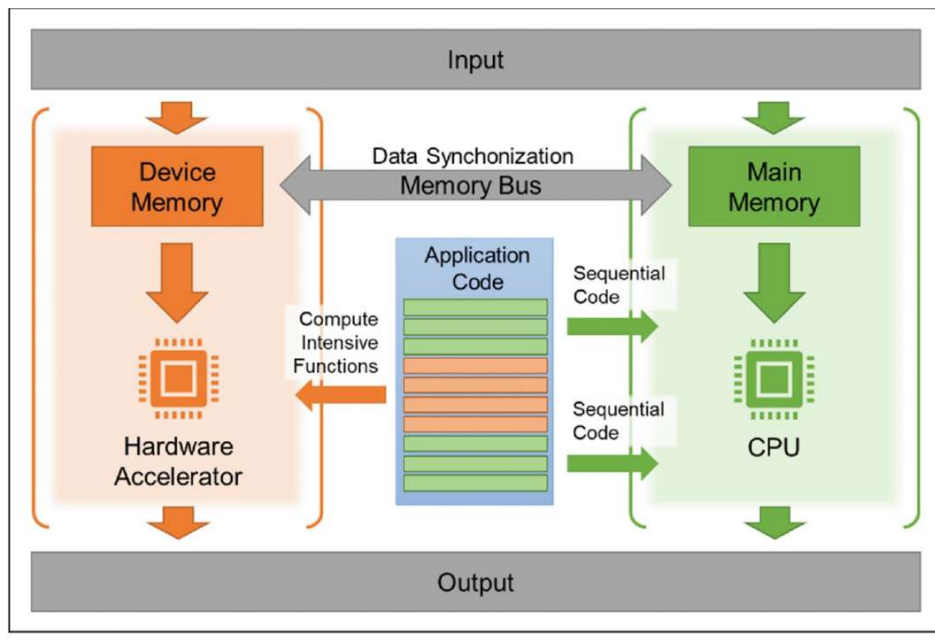
CUDA software → GPU hardware mapping

- **CUDA abstraction maps directly onto GPU hardware:**
 - CUDA thread → executes on a single CUDA core (FP32 unit)
 - Warp (32 threads) → scheduled as one unit by the warp scheduler; executes one instruction in SIMT lockstep
 - Thread block → assigned to one SM; shares that SM's shared memory
 - Grid of blocks → distributed across all SMs on the GPU
- **Implication for performance:**
 - Shared memory is per-SM: blocks cannot share data across SMs
 - Global memory (DRAM) is accessible by all threads but is slow
 - Maximising SM occupancy = keeping the warp scheduler busy
 - Warp divergence (branching within a warp) serializes lanes → avoid if/else on thread ID within a warp
- **Occupancy = active warps / max warps per SM**
 - Limited by register count, shared memory usage, and block size
 - Low occupancy → memory latency not hidden → idle compute units



Programming and execution model

A GPU is built around an **array of Streaming Multiprocessors (SMs)**. A **multithreaded program partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.

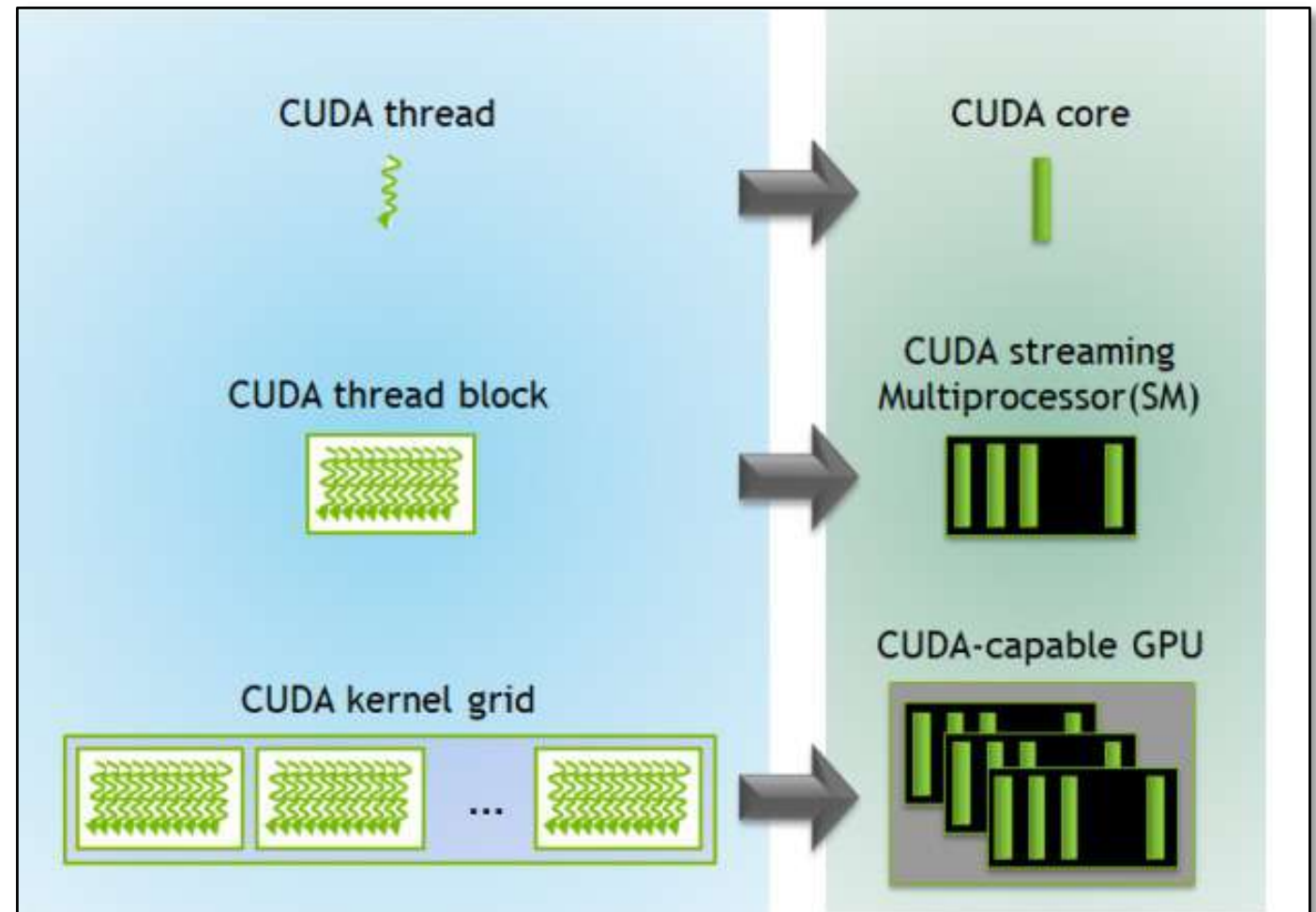
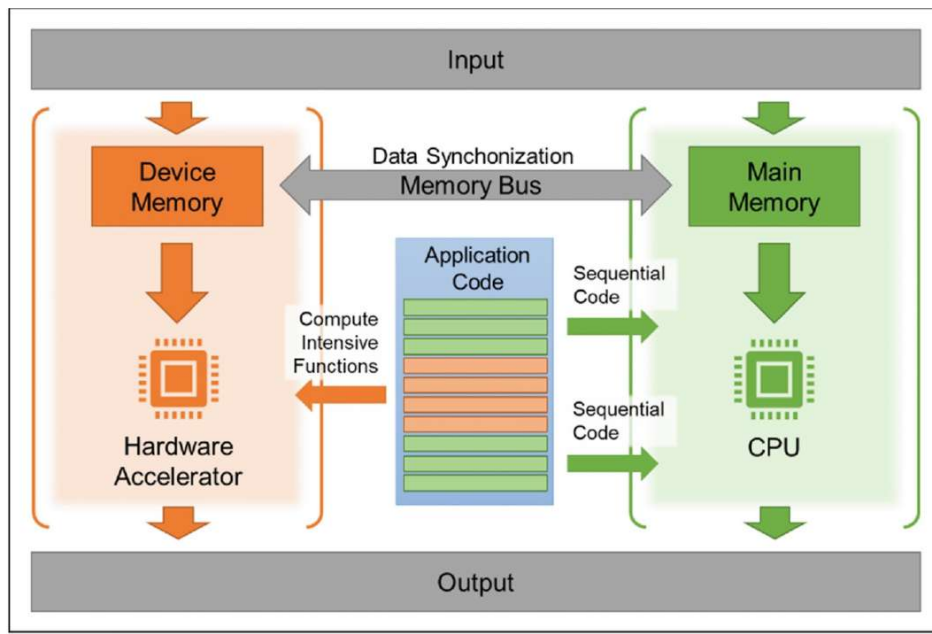


CUDA programming model assumes that the **CUDA threads execute on a physically separate device** that operates as a coprocessor to the host running the C++ program.

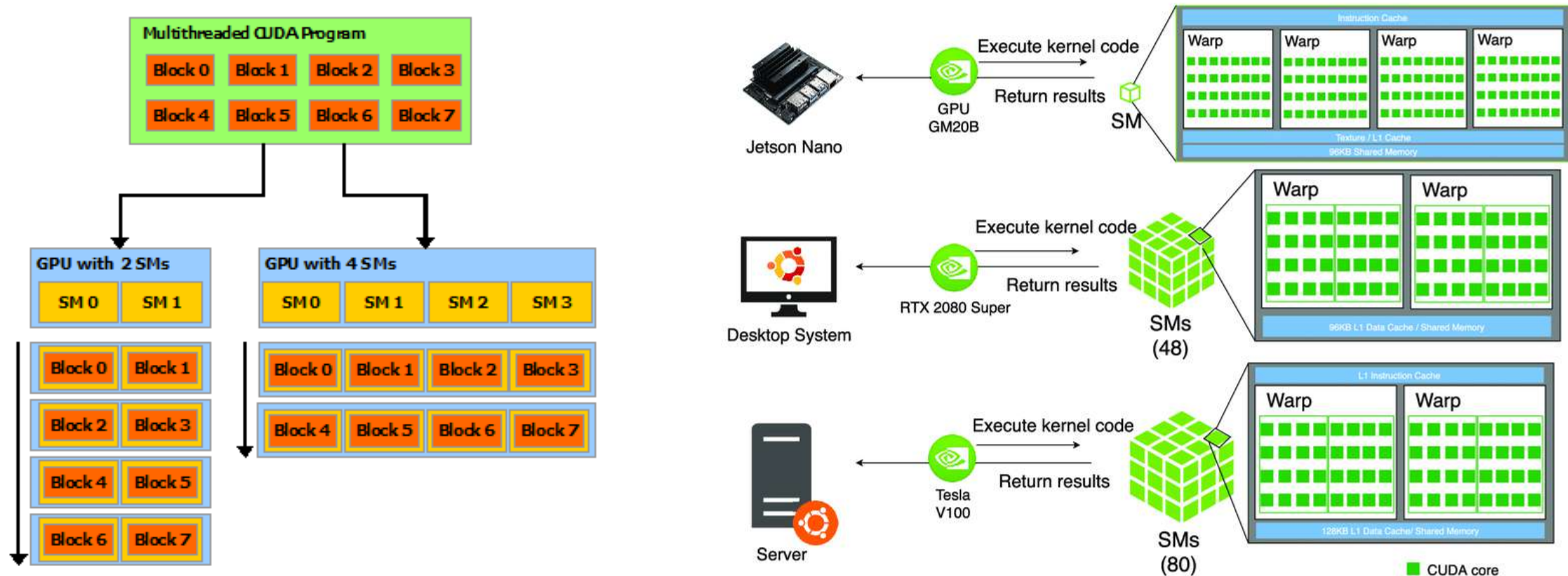
Serial code executes on the host while parallel code executes on the device.

Programming and execution model

A GPU is built around an **array of Streaming Multiprocessors (SMs)**. A multithreaded program is **partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.



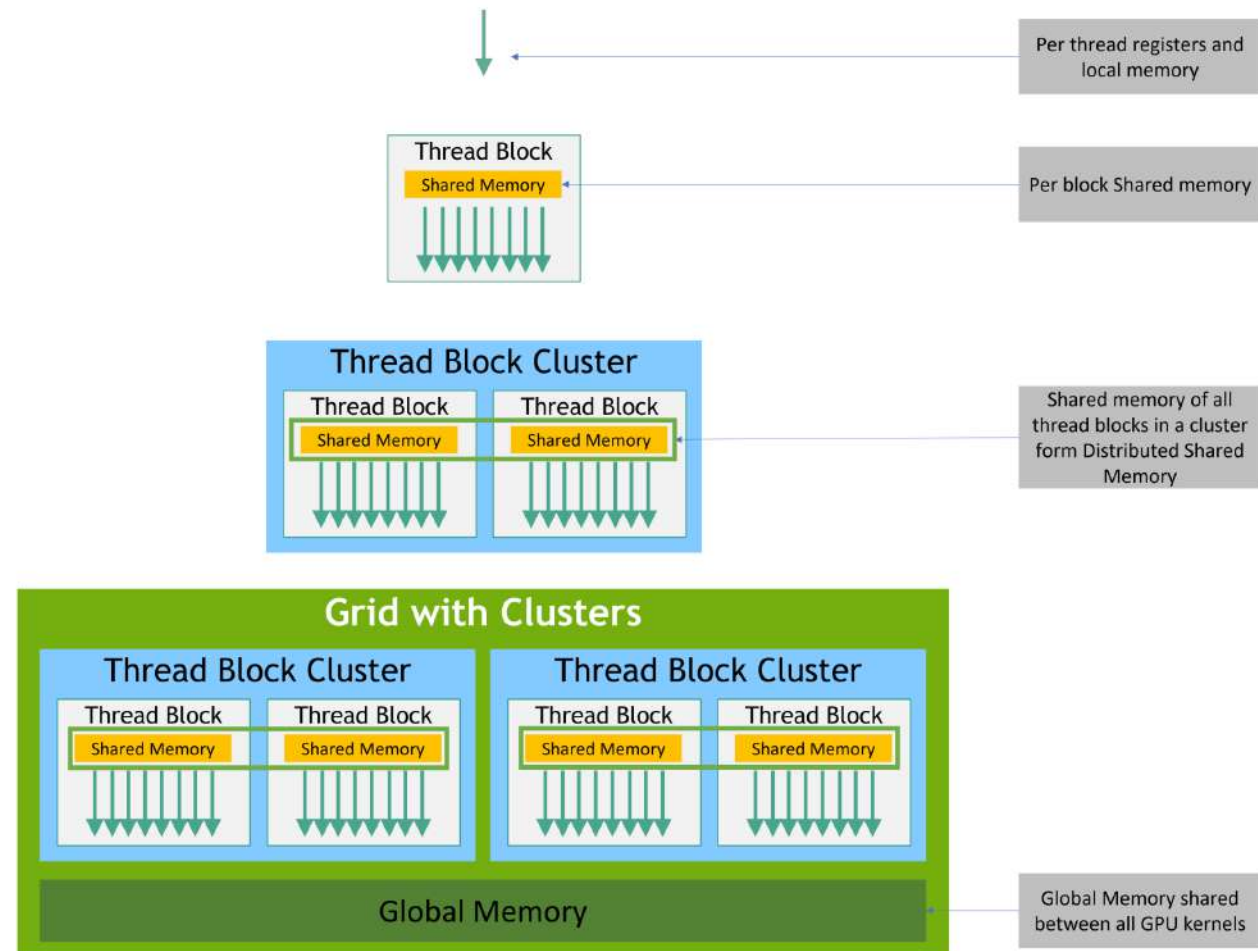
Automatic scalability



A GPU is built around an **array of Streaming Multiprocessors (SMs)**. A **multithreaded program is partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.

Memory hierarchy

- CUDA threads may access data from multiple memory spaces during their execution.
- **Each thread has private local memory.**
- **Each thread block has shared memory visible to all threads of the block and with the same lifetime as the block.**
- Thread blocks in a thread block cluster can perform read, write, and atomics operations on each other's shared memory.
- **All threads have access to the same global memory.**



Kernels (1)

- CUDA C++ extends C++ by allowing the programmer to define C++ functions, called **kernels**, that, when called, **are executed N times in parallel by N different CUDA threads, as opposed to only once like regular C++ functions.**
- A kernel is defined using the **__global__ declaration specifier** and the number of CUDA threads that execute that kernel for a given kernel call is specified using a new **<<<...>>> execution configuration** syntax.
- Each thread that executes the kernel is given a unique thread ID that is accessible within the kernel through built-in variables.

```
#include <iostream>

__global__ void kernel( void ) {

}

int main( void ) {
    kernel<<<1,1>>>();
    printf( "Hello, World!\n" );
    return 0;
}
```

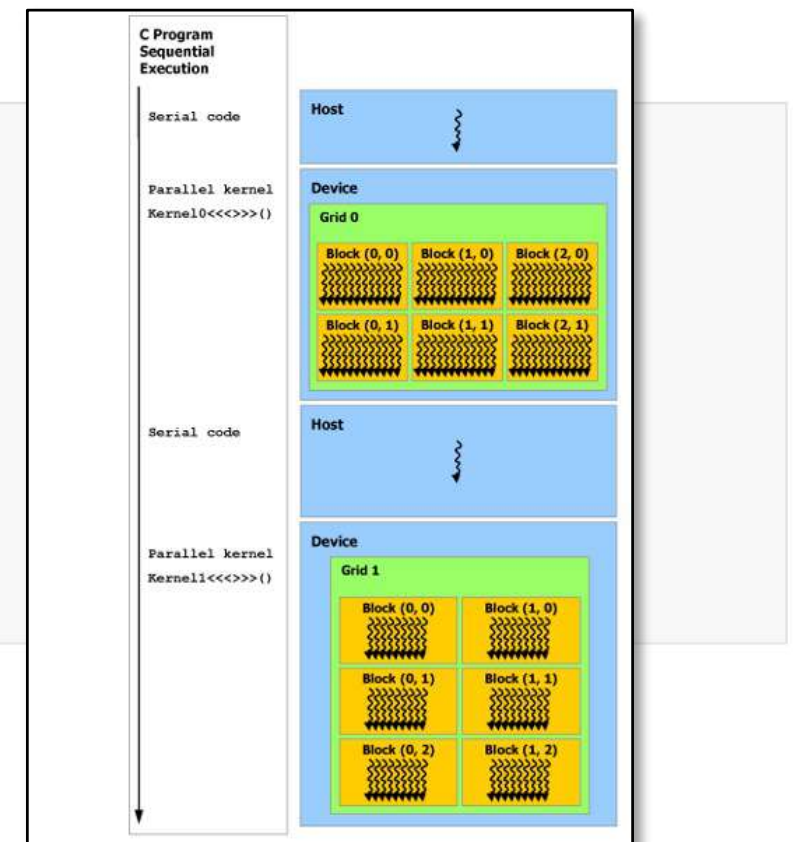
Kernels (2)

- As an illustration, the following sample code, using the built-in variable threadIdx, adds two vectors A and B of size N and stores the result into vector C.
- Single Instruction, Multiple Threads (SIMT)**

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

Here, each of the N threads that execute `VecAdd()` performs one pair-wise addition.





Christof Teuscher



teuscher@pdx.edu

www.teuscher-lab.com/teaching



GPU Memory Hierarchy

A100 TENSOR-CORE GPU

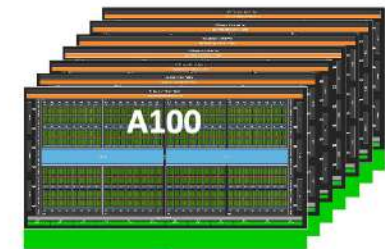
54 billion transistors in 7nm



Scale OUT

Multi-Instance GPU

7x



Scale UP

2x BW
3rd gen.
NVLINK

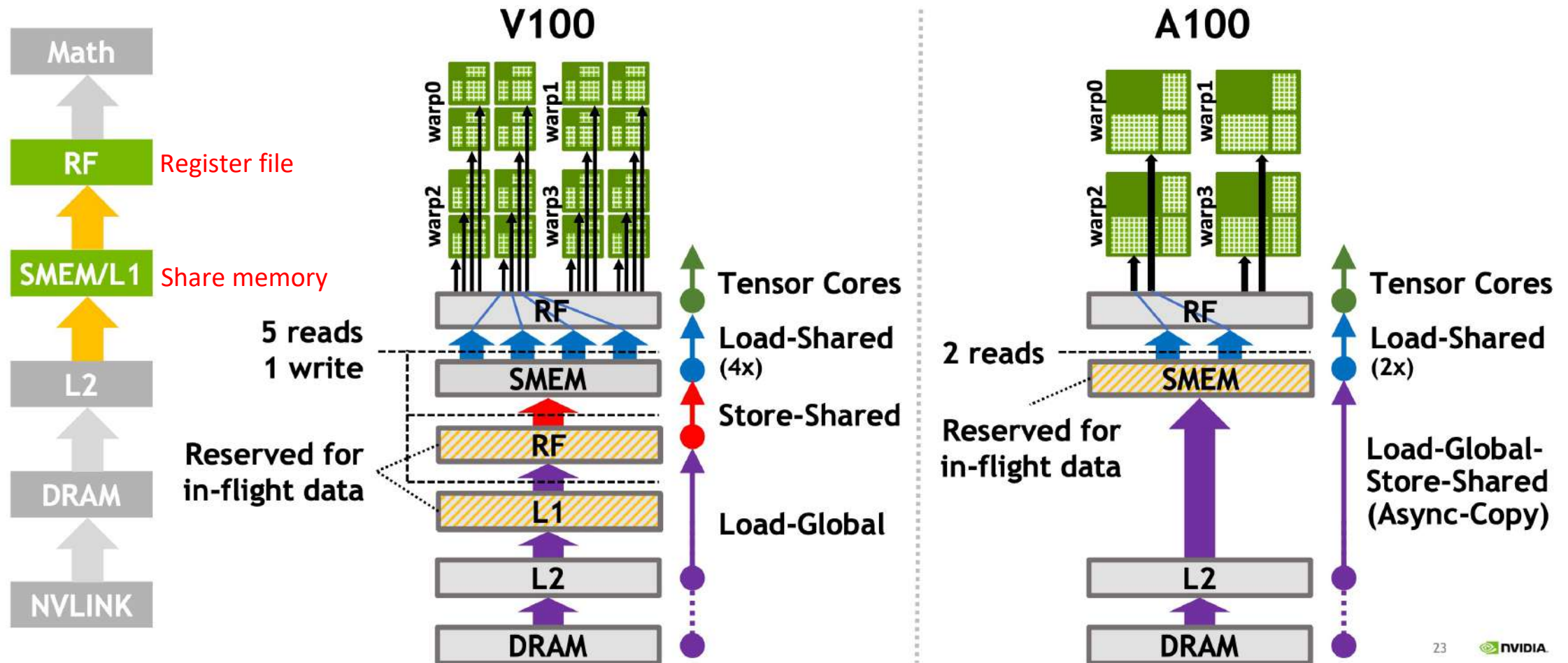
108 SMs
6912 CUDA Cores

40MB L2
6.7x capacity

1.56 TB/s HBM2
1.7x bandwidth

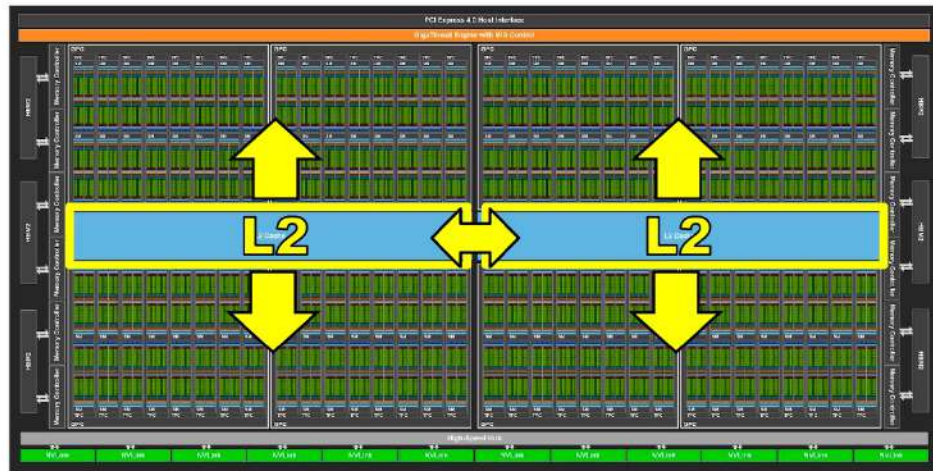
A100 data movement efficiency

3x SMEM/L1 bandwidth, 2x in-flight capacity



A100 data movement efficiency

Split L2 with
hierarchical crossbar -
2.3x increase in
bandwidth over V100,
lower latency

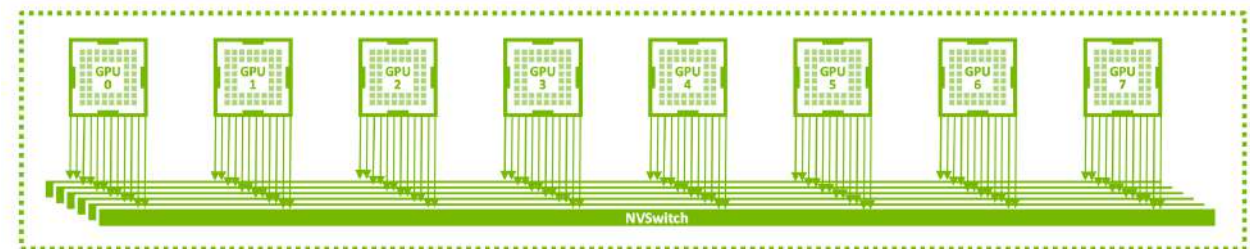
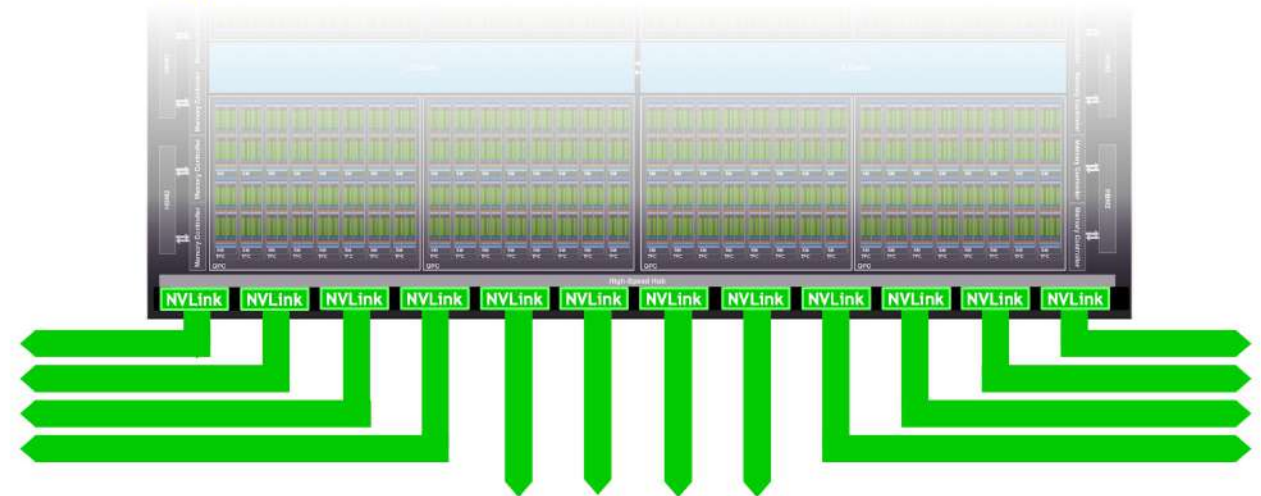


Third Generation NVLink

50 Gbit/sec per signal pair

12 links, 25 GB/s in/out, 600 GB/s total

2x vs. V100



Hardware consistency

Christof Teuscher

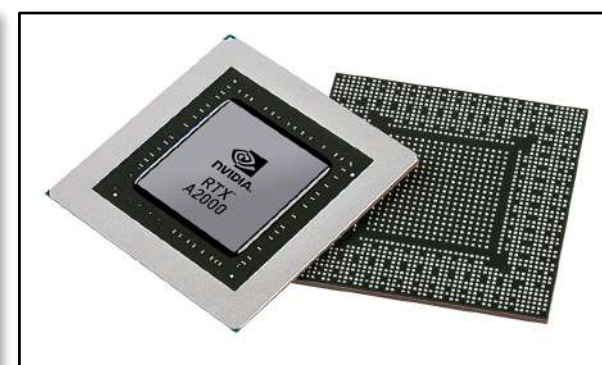
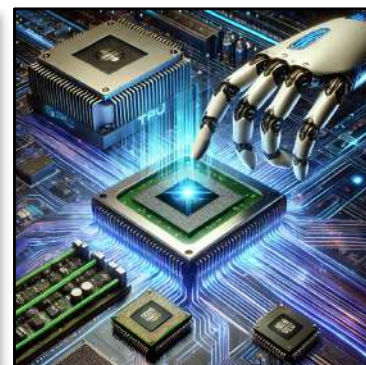
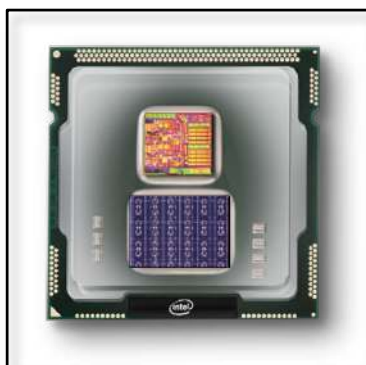
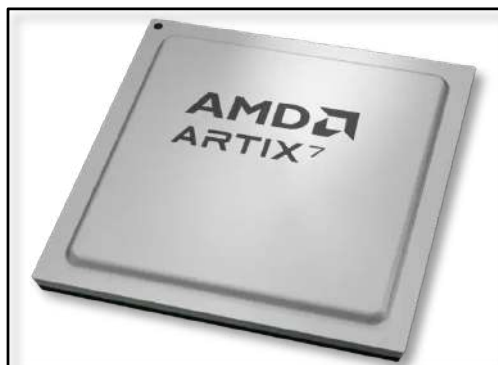
ECE 410/510: Hardware for AI and ML, Spring 2026

Week 3: Recap, outlook, and reminders

Portland State University
Department of Electrical and Computer Engineering (ECE)

www.teuscher-lab.com

teuscher@pdx.edu





Christof Teuscher



teuscher@pdx.edu

www.teuscher-lab.com/teaching



What did you learn last week?

Can you spend \$250k on tokens per year?



Pradeep Ravindra • 2nd

AI dude in Biotech | Former U.S. Army Intelligence | Penn Data ...

4d •

+ Follow

Jensen Huang said that a \$500k engineer is expected to spend \$250k in tokens per year.

Most people hear that and think - how the hell do you even spend that much in tokens?

You stop treating AI like a toy chatbot and start treating it like parallel engineering capacity.

The highest ROI for coding agents is:

- 1) Development loops
- 2) CI/CD [continuous integration/continuous operation]
- 3) Plan Mode

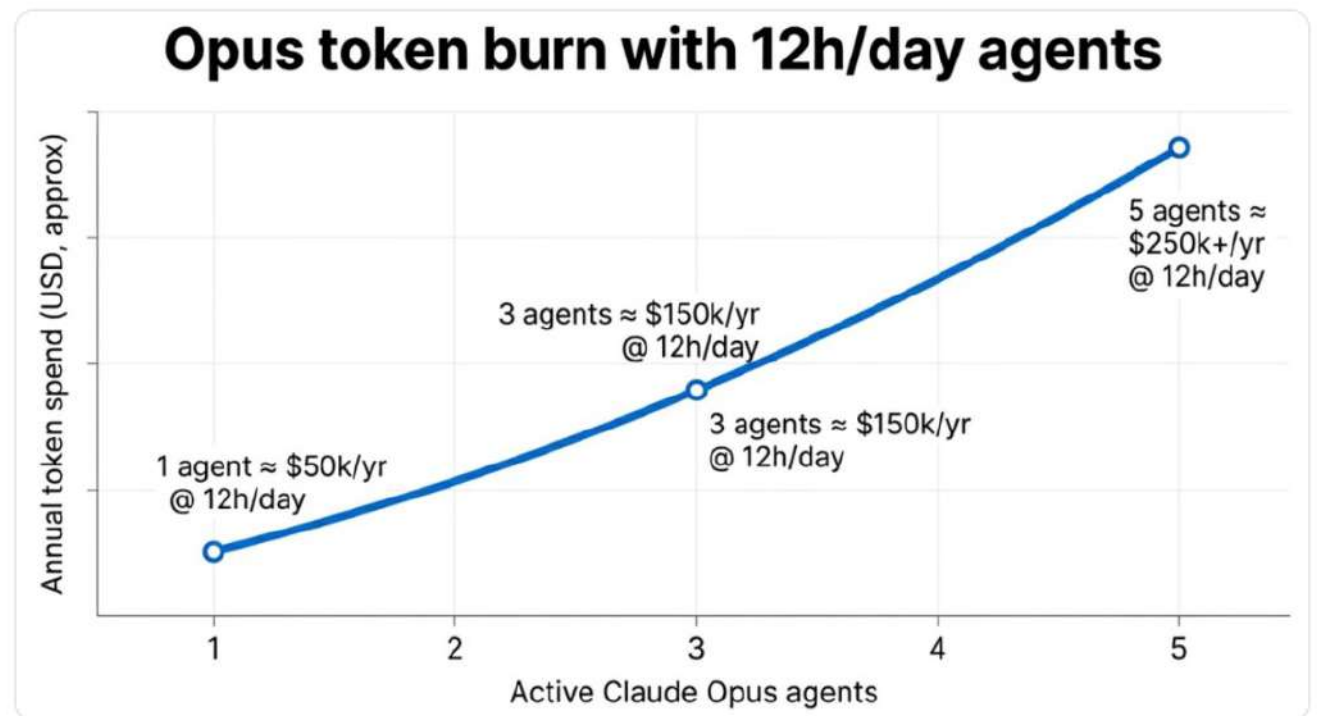
Now throw Git worktrees into the mix. One worktree per ticket, each with its own Claude Code session running in parallel.

Worktree A: feature branch

Worktree B: bugfix

Worktree C: migration in Plan Mode

Same engineer, multiple agents working at once. Suddenly, \$250k a year in tokens looks conservative.





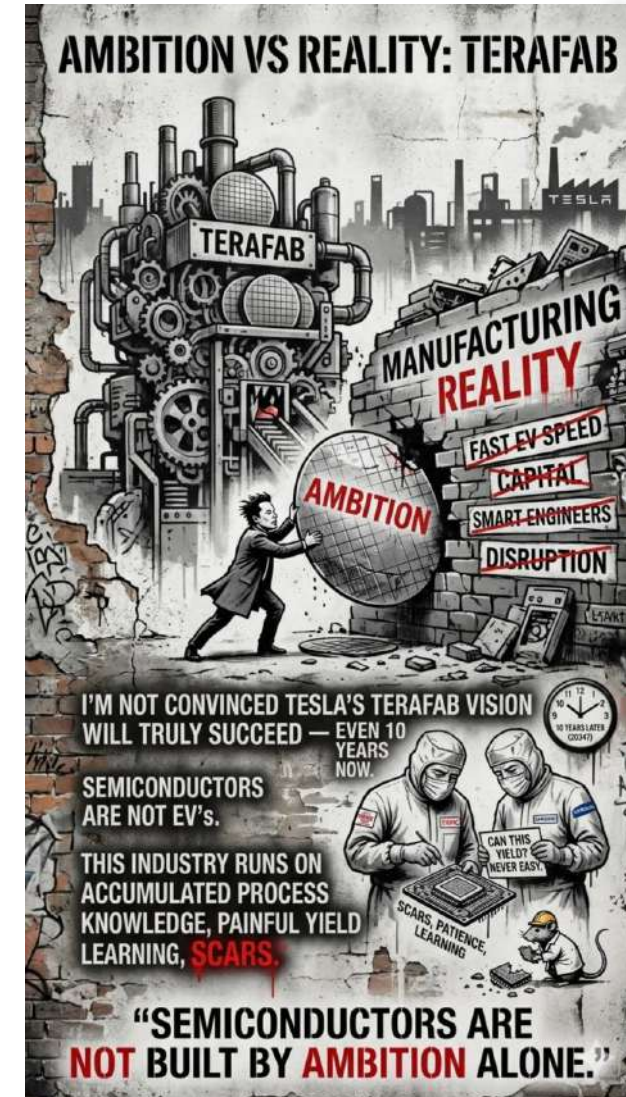
Updated 2 hours ago

Intel joins Musk's massive Terafab chipmaking project

→ Share

Edited by Emma W. Thorne, Editor at LinkedIn News

Intel [is joining](#) Terafab, the ambitious chipmaking project announced last month by Elon Musk. Shares in the California-based chip giant rose early Tuesday on the news. Terafab is a joint venture between Musk's companies SpaceX, xAI and Tesla that's meant to unite all the stages of the chipmaking process "[under one roof](#)," and [churn out chips](#) for Optimus robots and SpaceX satellites faster than other companies have been able to so far. Musk has touted the \$20 billion-plus project as "the most epic chip-building exercise in history by far."

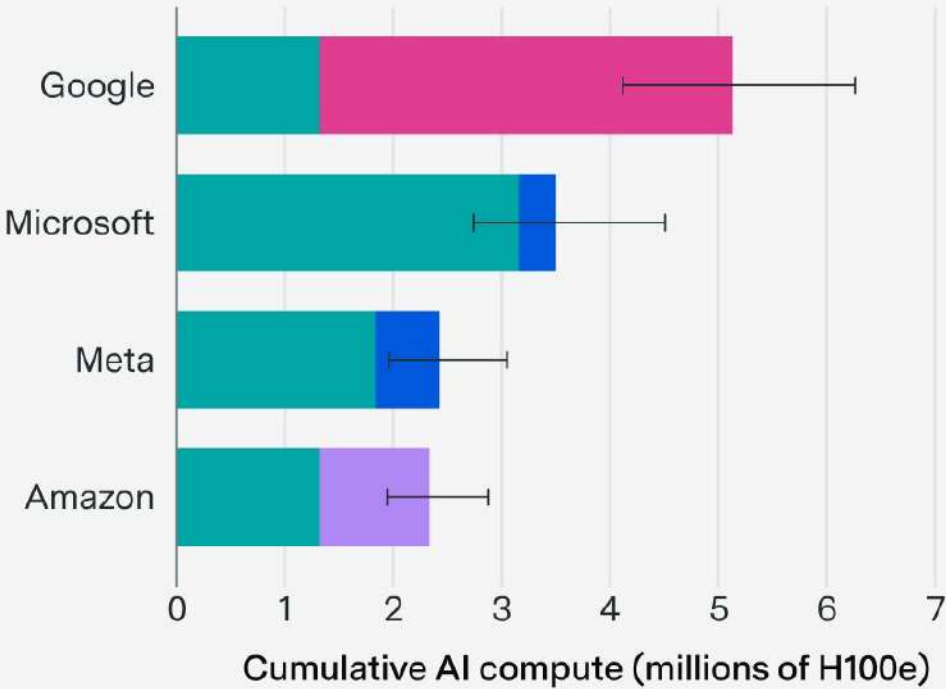


<https://www.linkedin.com/news/story/intel-joins-musks-massive-terafab-chipmaking-project-7160340>

Custom TPUs make Google the largest owner of AI compute

Cumulative chip acquired between Q1 2022 and Q4 2025, by chip manufacturer.

Nvidia Google TPU AMD AWS Trainium



Error bars correspond to 90% confidence intervals on total compute across all chip types.

Princeton researchers gave 12 AI models \$200K each and said: run a startup for a year. Hire people, find clients, manage cash, don't go bankrupt.

36 runs. A full year of simulation. Hundreds of business decisions in sequence. The results?

Only 5 out of 12 models turned a profit. The other 7 burned through everything. 47% of bankruptcies came down to one thing: the AI failed to detect an adversarial client.

Top 3 performers:

Claude Opus 4.6: \$1.27M in final funds. 0 bankruptcies in 3 runs. \$86 inference cost.

GLM-5: \$1.21M, also 0 bankruptcies. \$7.62 inference cost (11x cheaper).

GPT-5.4: over \$1M, 0 bankruptcies.

But the ranking isn't the interesting part. The decision-making patterns are. Models that survived focused on 1-2 key clients. Built trust. Deepened relationships. The ones that went bankrupt took on everyone, inflated headcount, and when 35% of clients turned out to be problematic, they had no resources left to recover.

Sound familiar? It's exactly what happens to real startups.

Here's what surprised me most: top-performing models kept a "scratchpad," writing out their reasoning before every decision. This turned out to be the single strongest predictor of survival. Not model size. Not speed. The habit of thinking before acting.

This is the first benchmark that measures not coding or math, but business reasoning over a long horizon. 3 out of 12 models turned \$200K into \$1M+. In one year.

AI can already run a business. Poorly, but it can. The question is who starts using it as a co-founder first.

Paper: arxiv.org/abs/2604.01212

arXiv > cs > arXiv:2604.01212

Search...

Help | Adv

Computer Science > Computation and Language

[Submitted on 1 Apr 2026]

YC-Bench: Benchmarking AI Agents for Long-Term Planning and Consistent Execution

Muyu He, Adit Jain, Anand Kumar, Vincent Tu, Soumyadeep Bakshi, Sachin Patro, Nazneen Rajani

As LLM agents tackle increasingly complex tasks, a critical question is whether they can maintain strategic coherence over long horizons: planning under uncertainty, learning from delayed feedback, and adapting when early mistakes compound. We introduce YC-Bench, a benchmark that evaluates these capabilities by tasking an agent with running a simulated startup over a one-year horizon spanning hundreds of turns. The agent must manage employees, select task contracts, and maintain profitability in a partially observable environment where adversarial clients and growing payroll create compounding consequences for poor decisions. We evaluate 12 models, both proprietary and open source, across 3 seeds each. Only three models consistently surpass the starting capital of \$200K, with Claude Opus 4.6 achieving the highest average final funds at \$1.27 M, followed by GLM-5 at \$1.21 M at 11x lower inference cost. Scratchpad usage, the sole mechanism for persisting information across context truncation, is the strongest predictor of success, and adversarial client detection is the primary failure mode, accounting for 47% of bankruptcies. Our analysis reveals that frontier models still fail through distinct failure modes such as over-parallelization, demonstrating the capability gaps for long-horizon performance. YC-Bench is open-source, reproducible, and configurable.

<https://arxiv.org/abs/2604.01212>

Introducing Project Glasswing: an urgent initiative to help secure the world's most critical software.

It's powered by our newest frontier model, Claude Mythos Preview, which can find software vulnerabilities better than all but the most skilled humans: <https://lnkd.in/eS-fu9BF>

We've partnered with Amazon Web Services, Apple, Broadcom, Cisco, CrowdStrike, Google, JPMorganChase, the Linux Foundation, Microsoft, NVIDIA, and Palo Alto Networks. Together we'll use Mythos Preview to help find and fix flaws in the systems on which the world depends.

Mythos Preview has already found thousands of high-severity vulnerabilities—including some in every major operating system and web browser.

Given the pace of AI progress, it won't be long before models this capable are widespread. But there are strong reasons for optimism: AI will also be invaluable for defensive work.

We do not plan to make Mythos Preview generally available. Our goal is to deploy Mythos-class models safely at scale, but first we need safeguards that reliably block their most dangerous outputs. We'll begin testing those safeguards with an upcoming Claude Opus model.

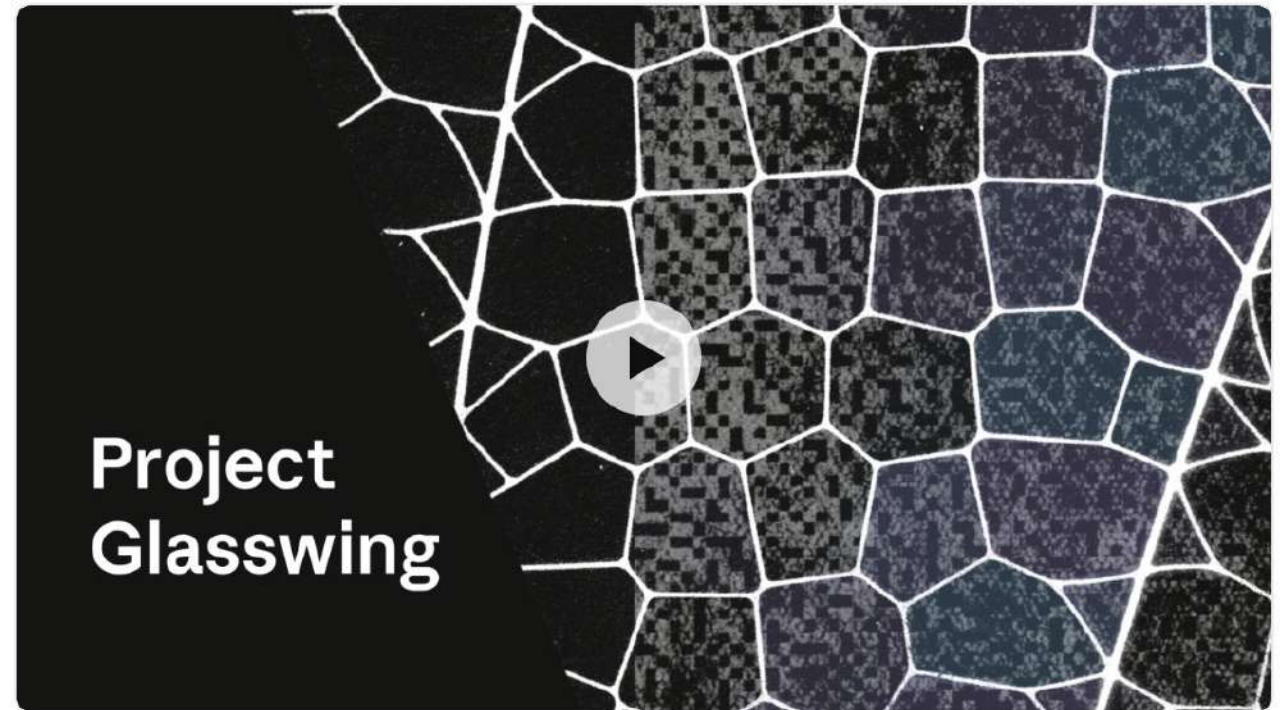
We're committing up to \$100M in Mythos Preview usage credits for our partners and over 40 other organizations that maintain critical software, including open-source projects. Anthropic will report back what we learn.

Project Glasswing is just a starting point. No organization can solve these cybersecurity problems alone: industry, open source, researchers, and governments all have essential roles to play.

You can read a detailed technical report on the software vulnerabilities and exploits discovered by Claude Mythos Preview here: <https://lnkd.in/eE7Kp9ZX>

The Claude Mythos Preview system card is available here: <https://lnkd.in/esH2-mfd>

www.teuscher-lab.com/teaching



Mythos Preview vs GPT-5.4 Pro

Only four benchmarks are reported in common between Anthropic's Mythos Preview system card and the GPT-5.4 Pro column of OpenAI's launch post. Here's the head-to-head.

BENCHMARKS IN COMMON

GPQA Diamond	Mythos		94.5	≈ tie +0.1 MYTHOS
	5.4 Pro		94.4	
Humanity's Last Exam no tools	Mythos		56.8	+14.1 MYTHOS
	5.4 Pro		42.7	
Humanity's Last Exam with tools	Mythos		64.7	+6.0 MYTHOS
	5.4 Pro		58.7	
BrowseComp	Mythos		86.9	+2.4 5.4 PRO
	5.4 Pro		89.3	

API PRICING

	Mythos Preview	GPT-5.4 Pro
Input	\$25 / MTok	\$30 / MTok
Output	\$125 / MTok	\$180 / MTok
Long context	same price	2× (\$60 in / \$270 out)
Access	Glasswing only	Public API

RPU – A Reasoning Processing Unit

Matthew Joseph Adiletta, Gu-Yeon Wei and David Brooks
Harvard University

Abstract—Large language model (LLM) inference performance is increasingly bottlenecked by the memory wall. While GPUs continue to scale raw compute throughput, they struggle to deliver scalable performance for memory bandwidth bound workloads. This challenge is amplified by emerging reasoning LLM applications, where long output sequences, low arithmetic intensity, and tight latency constraints demand significantly higher memory bandwidth. As a result, system utilization drops and energy per inference rises, highlighting the need for an optimized system architecture for scalable memory bandwidth.

To address these challenges we present the Reasoning Processing Unit (RPU), a chiplet-based architecture designed to address the challenges of the modern memory wall. RPU introduces: (1) A Capacity-Optimized High-Bandwidth Memory (HBM-CO) that trades capacity for lower energy and cost; (2) a scalable chiplet architecture featuring a bandwidth-first power and area provisioning design; and (3) a decoupled microarchitecture that separates memory, compute, and communication pipelines to sustain high bandwidth utilization. Simulation results show that RPU performs up to 45.3× lower latency and 18.6× higher throughput over an H100 system at ISO-TDP on Llama3-405B.

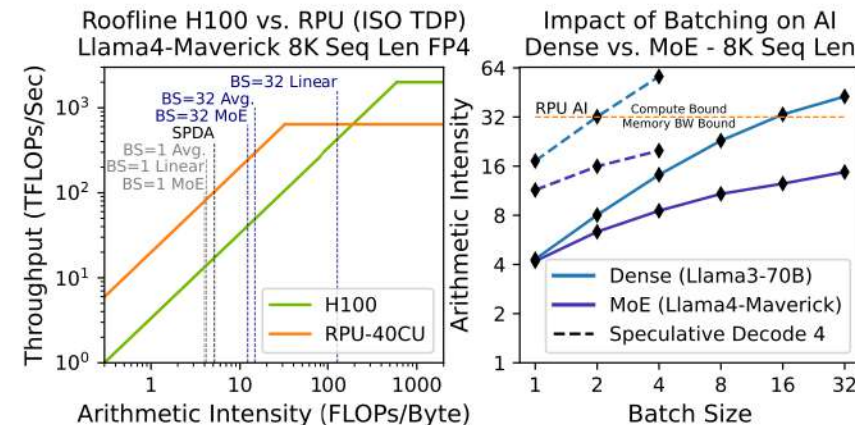


Fig. 1. RPU provides higher memory bandwidth than H100, which is required for low-latency decoding. Even up to BS=32, arithmetic intensity remains low, but requires the RPU to execute kernels which straddle the roofline.

System architects have responded to the decode bottleneck by cramming more HBM stacks into each package as the simplest way to chase bandwidth [14], [27], [61], [63]. How-

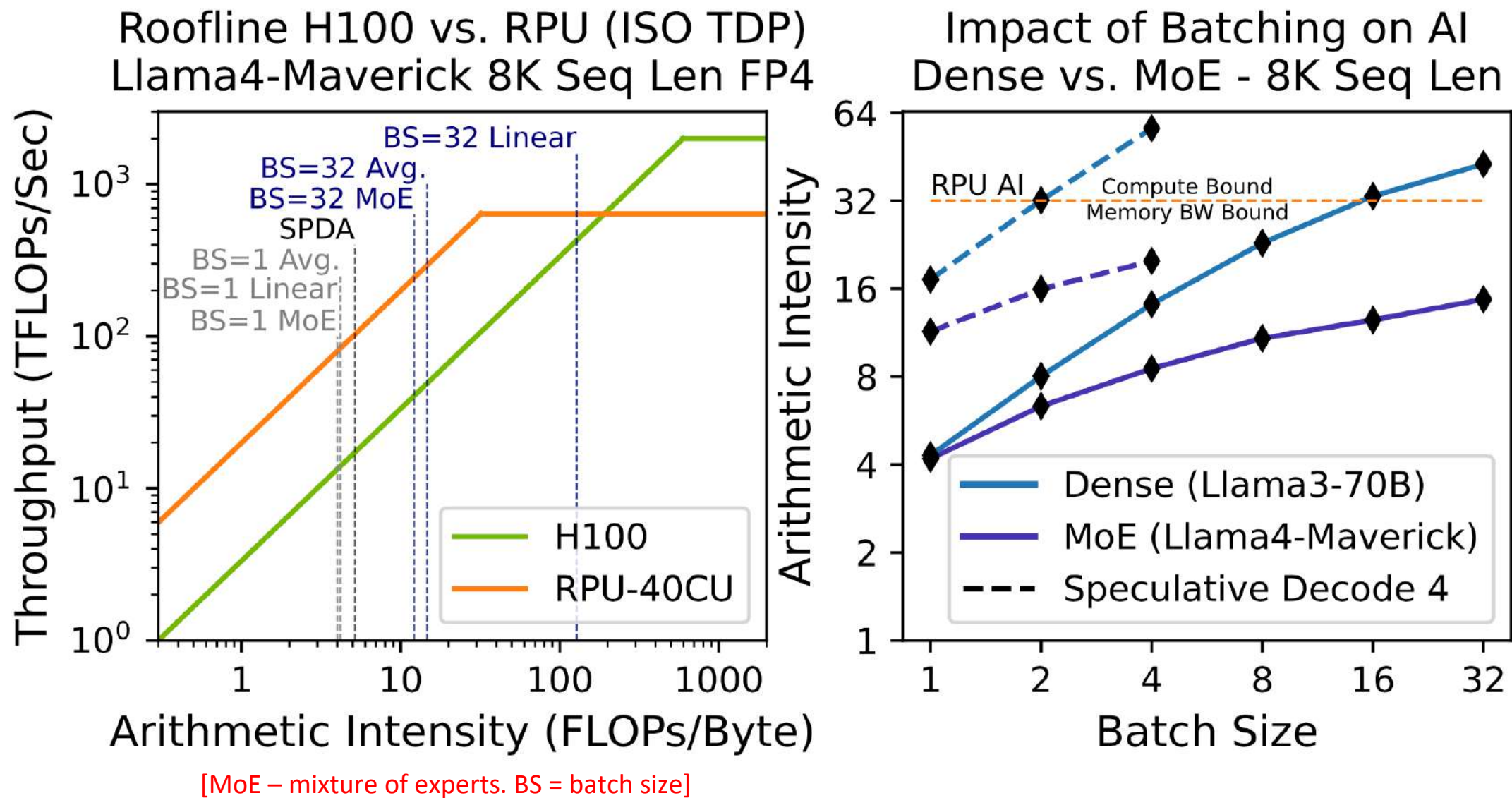


Fig. 1. RPU provides higher memory bandwidth than H100, which is required for low-latency decoding. Even up to BS=32, arithmetic intensity remains low, but requires the RPU to execute kernels which straddle the roofline.

Research Finds AI's Energy Use Is Driving Concern →

 **sambanova**

Products Solutions Developers Resources Company | Q EN ▾ [Start Building](#)

[← BACK TO RESOURCES](#)

News

[RDU = Reconfigurable Dataflow Unit]

SambaNova and Intel Announce Blueprint for Heterogeneous Inference: GPUs For Prefill, SambaNova RDUs for Decode, and Intel® Xeon® 6 CPUs for Agentic Tools

by SambaNova | 📅 April 8, 2026

<https://sambanova.ai/press/sambanova-announces-collaboration-with-intel-on-ai-solution>

<https://sambanova.ai/products/rdu-ai-chips>

Landscape: Delivering Premium Inference on the Largest Workloads			
What systems are used for inference?	Prefill: GPU/RDU Agnostic Decode: SambaNova SN50 CPU: Intel Xeon 6	Prefill: Nvidia Rubin GPU Decode: Rubin GPU & Groq 3 LPU CPU: Vera CPU	Prefill: GPU Agnostic Decode: Cerebras CS-3 CPU: RISC-based CPU
How many decode chips are required to deliver Premium Inference on trillion parameter models?	256 Chips SambaNova SN50	2000+ Chips Groq 3 LPU	Not possible Cerebras CS-3
What data centers does this system fit in?	Plugs into existing data centers — requires just 30 kW per rack	Requires new liquid-cooled data center facility — 1 MW+	Not applicable
How well does the hardware adapt to changing premium workloads?	High utilization	Low utilization	Not applicable
What is the software coverage for agentic tools?	 Intel Xeon 6	 Nvidia Vera CPU	 RISC-based CPU

Bloomberg

About us

Careers

Stories

Find jobs



AI Accelerator Market Looks Set to Exceed \$600 Billion by 2033, Driven by Hyperscale Spending and ASIC Adoption, According to Bloomberg Intelligence

GPU Market to Reach \$486 Billion; ASIC Market Heads to \$118 Billion

The research projects that GPUs will maintain their dominant position, capturing 81% of the AI accelerator market by 2033 with revenues growing at a 14% compound annual rate to reach approximately \$486 billion. Nvidia is expected to hold 70-75% of the AI accelerator market through 2030, with AMD positioned as the credible second source with a projected minimum 10% share by decade's end.

Meanwhile, custom ASICs are experiencing explosive growth, with the market projected to expand at a 27% compound annual rate to reach \$118 billion by 2033, up from just 8% of the total market in 2024 to 19% by 2033. Broadcom and Marvell are leading this transformation – Broadcom commanding 60-80% of the AI ASIC market through partnerships with Google, Meta and OpenAI, and Marvell holding a 20-25% share through key design wins with Amazon Web Services and Microsoft.

Christof Teuscher

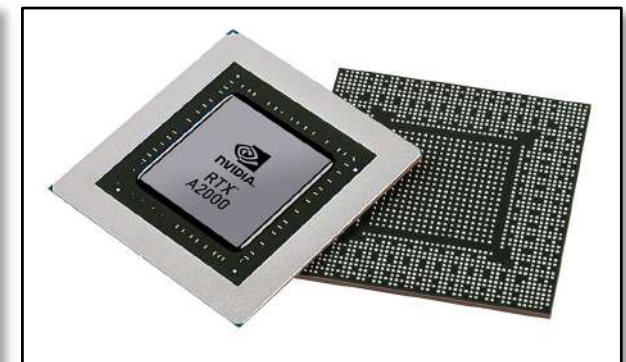
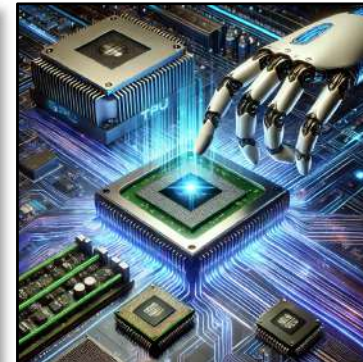
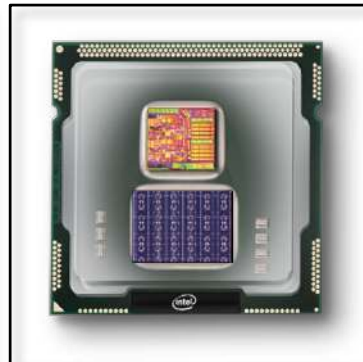
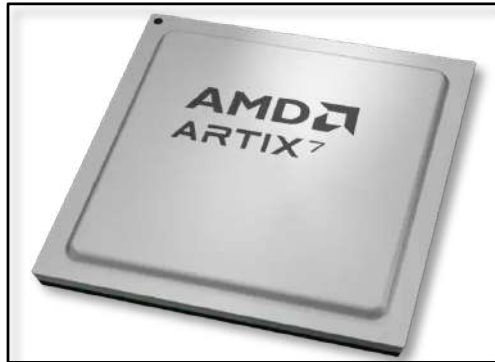
ECE 410/510: Hardware for AI and ML, Spring 2026

Codefest #3: Traffic Analysis and CUDA

Portland State University
Department of Electrical and Computer Engineering (ECE)

www.teuscher-lab.com

teuscher@pdx.edu





Today

CMAN

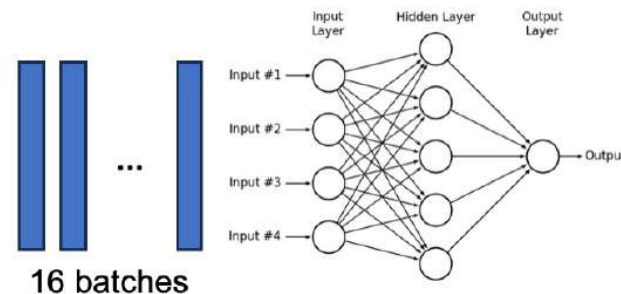
DRAM traffic analysis: naive vs. tiled matrix multiply

CLLM

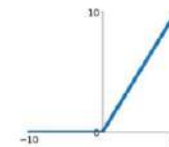
CUDA GEMM: naive vs. tiled with roofline placement

COPT

Optional challenge (+ 1 token): forward pass of a neural network on the GPU



ReLU
 $\max(0, x)$





Next week

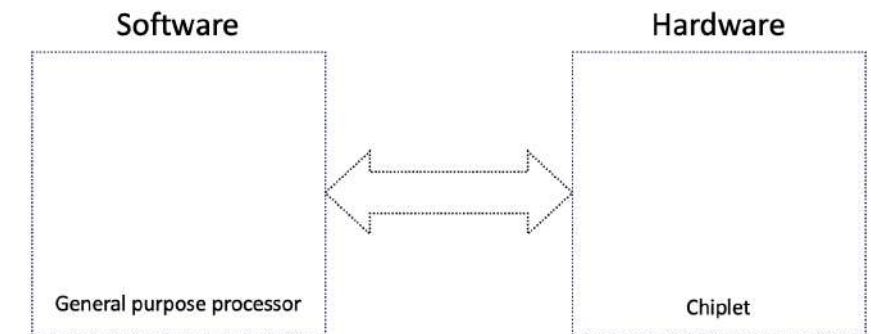
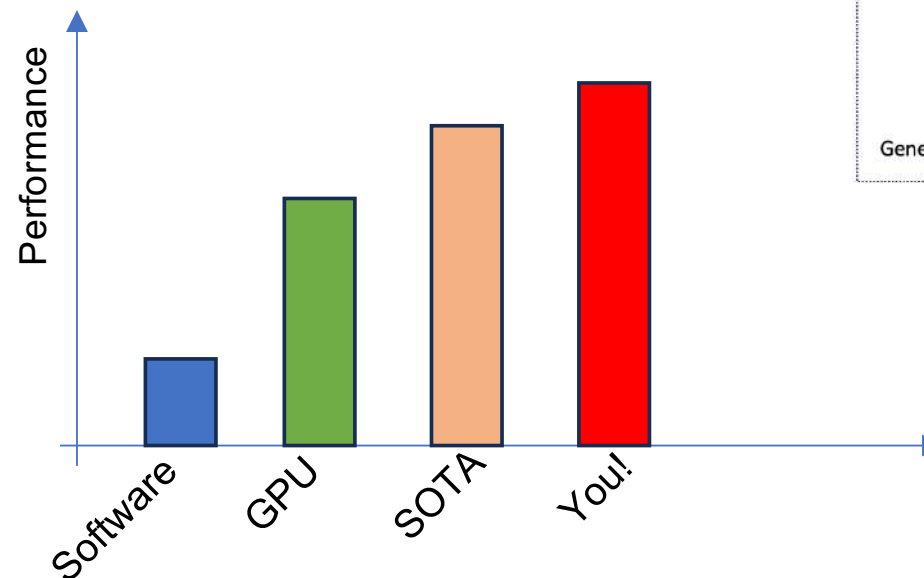
- We'll start with implementing the AI accelerator

- PyMTL (Mamba)
- PyRTL
- Cocotb
- MyHDL
- pyUVM
- yosys
- ...
- Cadence
- Synopsis

Continue to work toward M2

- Refine your ideas
- Do more profiling
- Run your code on GPU
- Re-consider the SW/HW boundary
- Compare with SOTA
- Needed precisions?

M2	Sun, May 3	First working simulation: HDL description of the compute core with a testbench demonstrating correct output for a representative input. Interface module in HDL with testbench exercising at least one write and one read/response transaction. Precision choice documented (FP32 / FP16 / BF16 / INT8 / other) with quantization error analysis if applicable.
----	------------	---



Total execution time (1)

```
import time

start_time = time.time()

# Code to be timed
def my_function():
    result = 0
    for i in range(1000000):
        result += i
    return result

my_function()

end_time = time.time()
execution_time = end_time - start_time

print(f"Execution time: {execution_time} seconds")
```

Python

```
import time

start_time = time.perf_counter()

# Code to be timed
def my_function():
    result = 0
    for i in range(1000000):
        result += i
    return result

my_function()

end_time = time.perf_counter()
execution_time = end_time - start_time

print(f"Execution time: {execution_time} seconds")
```

For more precise timing, especially for short durations, `time.perf_counter()` is recommended. It provides the highest available resolution to measure a short duration.

Total execution time (2)

Python

```
import timeit

# Code to be timed as a string
code_to_time = """
def my_function():
    result = 0
    for i in range(1000000):
        result += i
    return result
my_function()
"""

execution_time = timeit.timeit(code_to_time, number=1)

print(f"Execution time: {execution_time} seconds")
```

The timeit module is specifically designed for measuring the execution time of small code snippets.

Total execution time (3)

```
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.1457672119140625e-06 seconds
Execution time 2: 5.489999999885864e-07 seconds
Execution time 3: 6.220000000056736e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.1457672119140625e-06 seconds
Execution time 2: 4.089999999790095e-07 seconds
Execution time 3: 5.87999999958141e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.86102294921875e-06 seconds
Execution time 2: 2.548999999905866e-06 seconds
Execution time 3: 6.570000000150067e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 9.5367431640625e-07 seconds
Execution time 2: 5.140000000070089e-07 seconds
Execution time 3: 5.260000000006926e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.1457672119140625e-06 seconds
Execution time 2: 5.240000000017453e-07 seconds
Execution time 3: 5.35999999995429e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 1.9073486328125e-06 seconds
Execution time 2: 5.469999999896391e-07 seconds
Execution time 3: 6.000000000172534e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 9.5367431640625e-07 seconds
Execution time 2: 4.67000000003992e-07 seconds
Execution time 3: 5.95999999916032e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 1.1920928955078125e-06 seconds
Execution time 2: 5.990000000177798e-07 seconds
Execution time 3: 6.559999999877775e-07 seconds
```

Total execution time (4)

```
# Execution time
start_time1 = time.time()
start_time2 = time.perf_counter()
start_time3 = timeit.default_timer()

repeats = 10

if __name__ == "__main__":
    import sys

    for r in range(repeats):

        if len(sys.argv) != 2:
            print("Usage: python qr_recognizer.py <image_path>")
            sys.exit(1)

        image_path = sys.argv[1]
        recognizer = QRCodeRecognizer()

        try:
            result = recognizer.recognize(image_path)
            print("QR Code Recognition Result:")
            print(result)
        except Exception as e:
            print(f"Error: {e}")

    end_time1 = time.time()
    end_time2 = time.perf_counter()
    execution_time1 = end_time1 - start_time1
    execution_time2 = end_time2 - start_time2
    execution_time3 = timeit.default_timer() - start_time3

    print(f"Execution time 1: {execution_time1} seconds")
    print(f"Execution time 2: {execution_time2} seconds")
    print(f"Execution time 3: {execution_time3} seconds")
```

```
Execution time 1: 1.8990371227264404 seconds
Execution time 2: 1.8990448829999997 seconds
Execution time 3: 1.8990454230000002 seconds
```

Code analysis (1)

Static Code Analysis Tools

1. Using PyCallGraph

PyCallGraph is a tool that creates call graph visualizations for Python applications:

```
python

# Install with: pip install pycallgraph graphviz
from pycallgraph import PyCallGraph
from pycallgraph.output import GraphvizOutput

def analyze_qr_code():
    with PyCallGraph(output=GraphvizOutput()):
        recognizer = QRCodeRecognizer()
        result = recognizer.recognize("path_to_qr_image.png")
        print(result)

analyze_qr_code()
```

2. Using pyan for Static Analysis

Pyan analyzes Python code and generates approximate call graphs:

```
python

# Install with: pip install pyan
# Usage from command line
# pyan qr_recognizer.py --dot > qr_graph.dot
# dot -Tpng qr_graph.dot -o qr_graph.png
```


Code analysis (2)

Runtime Analysis Tools

3. Using cProfile with Visualization

```
python

import cProfile
import pstats
from pstats import SortKey

# Profile the execution
cProfile.run('recognizer.recognize("path_to_qr_image.png")', '

# Analyze the results
p = pstats.Stats('qr_stats')
p.strip_dirs().sort_stats(SortKey.CUMULATIVE).print_stats(10)

# For visualization, you can use gprof2dot
# pip install gprof2dot
# gprof2dot -f pstats qr_stats | dot -Tpng -o qr_profile.png
```

4. Using SnakeViz for Interactive Visualization

```
python

# Install with: pip install snakeviz
# Run profiling
import cProfile
cProfile.run('recognizer.recognize("path_to_qr_image.png")', '

# Then from command line:
# snakeviz qr_stats
```

Code analysis (3)

Performance Visualization

6. Visualize Performance Bottlenecks

To visualize execution time of different components:

```
python

import time
import numpy as np
import matplotlib.pyplot as plt

def benchmark_qr_components(image_path, iterations=10):
    recognizer = QRCodeRecognizer()
    components = {}

    # Benchmark image loading
    start = time.time()
    for _ in range(iterations):
        img = recognizer.load_image(image_path)
        components["load_image"] = (time.time() - start) / iterations

    # Benchmark thresholding
    start = time.time()
```

Using Specialized Tools

For more advanced analysis, consider these specialized tools:

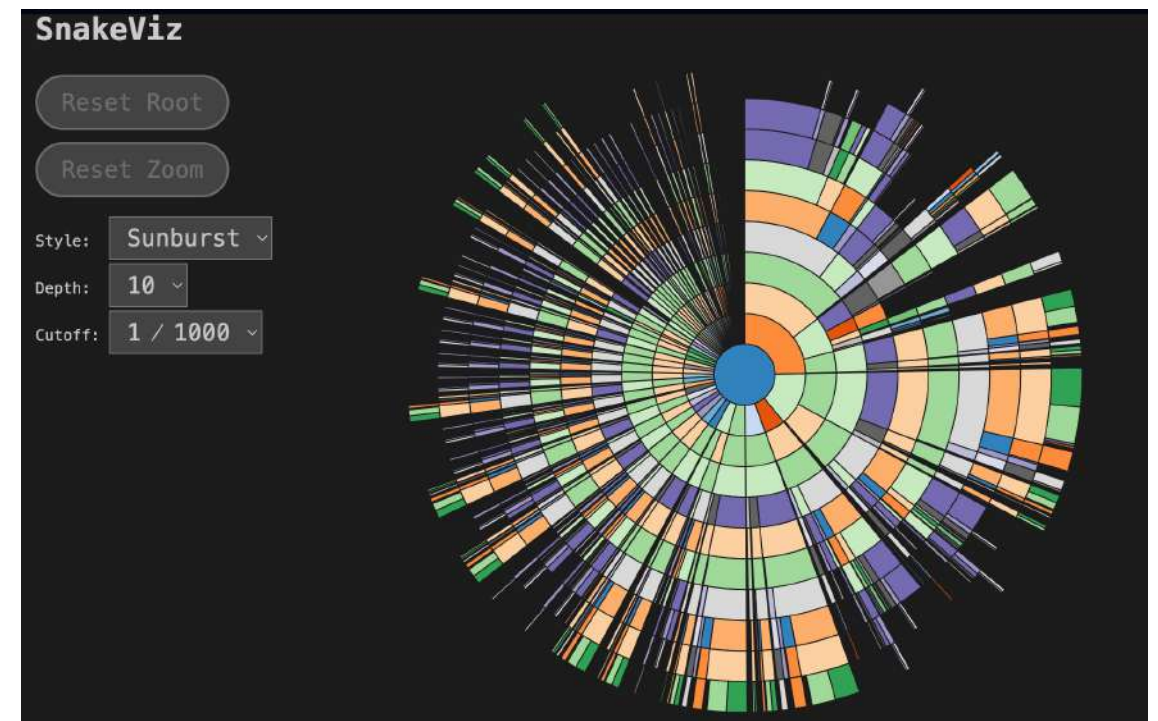
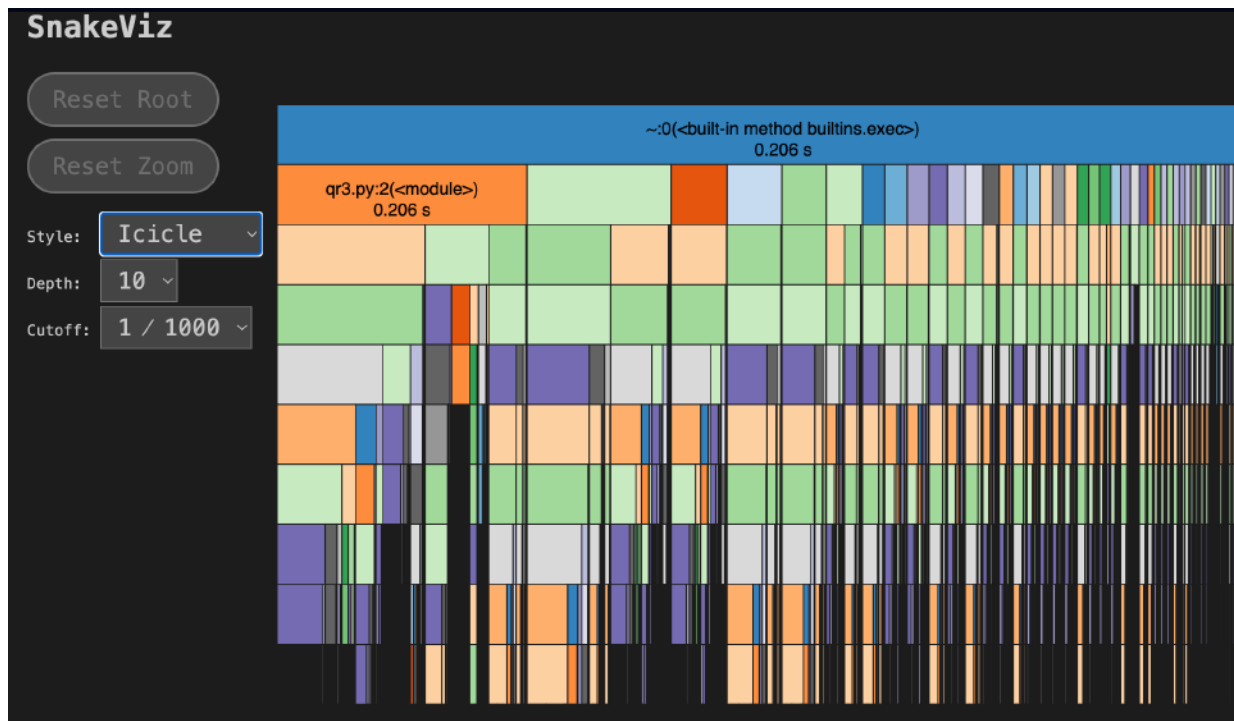
1. **Doxygen** - Can generate call graphs if properly configured
2. **CodeViz** - Works with C/C++ extensions but can be used on some Python projects
3. **Memray** - For memory profiling with visualization capabilities
4. **Py-Spy** - For sampling profiler that can generate flame graphs

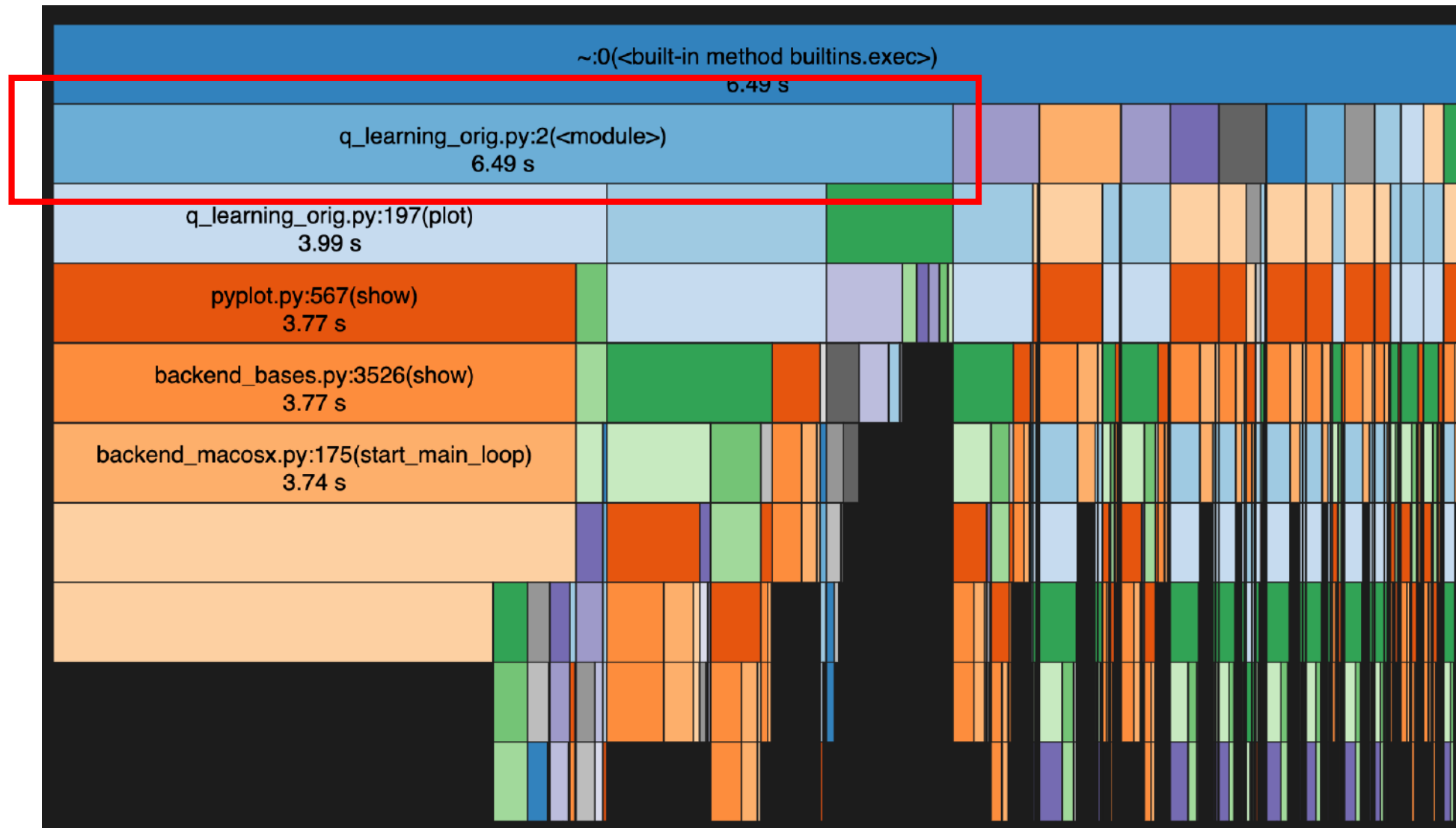
Each of these methods offers different insights into your code's structure and performance. Combining a few of them will give you the most comprehensive understanding of your QR code recognizer's data and control flow.



cProfile and SnakeViz

1. `pip install snakeviz`
2. `python3 -m cProfile -o output.prof qr_recognizer.py qr.png`
3. `snakeviz output.prof`





ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
37/36	0.03897	0.001082	0.04111	0.001142	~:0(<built-in method _imp.create_dynamic>)
153	0.01551	0.0001014	0.01551	0.0001014	~:0(<built-in method marshal.loads>)
1849	0.01443	7.803e-06	0.01443	7.803e-06	qr3.py:105(_is_potential_finder_pattern)
165	0.008252	5.001e-05	0.008252	5.001e-05	~:0(<method 'read' of '_io.BufferedReader' objects>)
153	0.006271	4.099e-05	0.006271	4.099e-05	~:0(<built-in method io.open_code>)
809	0.005534	6.84e-06	0.005534	6.84e-06	~:0(<built-in method posix.stat>)
390/389	0.00524	1.347e-05	0.02292	5.893e-05	~:0(<built-in method builtins.__build_class__>)
20	0.004723	0.0002362	0.008198	0.0004099	enum.py:179(__new__)
1	0.003492	0.003492	0.005462	0.005462	histograms.py:678(histogram)
388	0.002841	7.323e-06	0.01371	3.534e-05	<frozen importlib._bootstrap_external>:1438(find_spec)
65	0.00221	3.4e-05	0.00221	3.4e-05	~:0(<built-in method posix.getcwd>)

- **ncalls:** Total number of calls to the function. If there are two numbers, that means the function recursed and the first is the total number of calls and the second is the number of primitive (non-recursive) calls.
- **tottime:** Total time spent in the function, not including time spent in calls to sub-functions
- **percall:** `tottime` divided by `ncalls`
- **cumtime:** Cumulative time spent in this function and all sub-functions
- **percall:** `cumtime` divided by `ncalls`
- **filename:lineno(function):** File name and line number where the function is defined, and the function's name

line_profiler

1. kernprof -l q_learning_profiling.py
2. python3 -m line_profiler -rmt "q_learning_profiling.py.lprof"



```
#Q-learning Algorithm
@profile
def Q_Learning(self, episodes):
    x = 0
    #iterate through best path for each episode
    while(x < episodes):
        #check if state is end
        if self.isEnd:
            #get current reward and add to array for plot
            reward = self.State.getReward()
            self.rewards += reward
            self.plot_reward.append(self.rewards)

            #get state, assign reward to each Q_value in state
            i,j = self.State.state
            for a in self.actions:
                self.new_Q[(i,j,a)] = round(reward,3)

            #reset state
            self.State = State()
            self.isEnd = self.State.isEnd

            #set rewards to zero and iterate to next episode
            self.rewards = 0
            x+=1
        else:
            #set to arbitrary low value to compare net state actions
            mx_nxt_value = -10
            #get current state, next state, action and current reward
            next_state, action = self.Action()
            i,j = self.State.state
            reward = self.State.getReward()
            #add reward to rewards for plot
            self.rewards +=reward
```

Total time: 1.91771 s
 File: q_learning_profiling.py
 Function: Q_Learning at line 141

Line #	Hits	Time	Per Hit	% Time	Line Contents
141					@profile
142					def Q_Learning(self, episodes):
143	1	1.0	1.0	0.0	x = 0
144					#iterate through best path for each episode
145	94259	24463.0	0.3	1.3	while(x < episodes):
146					#check if state is end
147	94258	22655.0	0.2	1.2	if self.isEnd:
148					#get current reward and add to array for plot
149	10000	13540.0	1.4	0.7	reward = self.State.getReward()
150	10000	3810.0	0.4	0.2	self.rewards += reward
151	10000	5266.0	0.5	0.3	self.plot_reward.append(self.rewards)
152					
153					#get state, assign reward to each Q_value in state
154	10000	3039.0	0.3	0.2	i,j = self.State.state
155	50000	12719.0	0.3	0.7	for a in self.actions:
156	40000	26152.0	0.7	1.4	self.new_Q[(i,j,a)] = round(reward,3)
157					
158					#reset state
159	10000	9741.0	1.0	0.5	self.State = State()
160	10000	3885.0	0.4	0.2	self.isEnd = self.State.isEnd
161					
162					#set rewards to zero and iterate to next episode
163	10000	2903.0	0.3	0.2	self.rewards = 0
164	10000	3693.0	0.4	0.2	x+=1

Line #	Hits	Time	Per Hit	% Time	Line Contents
165					<code>else:</code>
166					<code>#set to arbitrary low value to compare net state actions</code>
167	84258	19736.0	0.2	1.0	<code>mx_nxt_value = -10</code>
168					<code>#get current state, next state, action and current reward</code>
169	84258	528816.0	6.3	27.6	<code>next_state, action = self.Action()</code>
170	84258	25683.0	0.3	1.3	<code>i,j = self.State.state</code>
171	84258	108610.0	1.3	5.7	<code>reward = self.State.getReward()</code>
172					<code>#add reward to rewards for plot</code>
173	84258	33549.0	0.4	1.7	<code>self.rewards +=reward</code>
174					
175					<code>#iterate through actions to find max Q value for action based on next state action</code>
176	421290	115314.0	0.3	6.0	<code>for a in self.actions:</code>
177	337032	111121.0	0.3	5.8	<code>nxtStateAction = (next_state[0], next_state[1], a)</code>
178	337032	272470.0	0.8	14.2	<code>q_value = (1-self.alpha)*self.Q[(i,j,action)] + self.alpha*(reward + self.gamma*...</code>
179					
180					<code>#find largest Q value</code>
181	337032	87482.0	0.3	4.6	<code>if q_value >= mx_nxt_value:</code>
182	212265	47640.0	0.2	2.5	<code>mx_nxt_value = q_value</code>
183					
184					<code>#next state is now current state, check if end state</code>
185	84258	95663.0	1.1	5.0	<code>self.State = State(state=next_state)</code>
186	84258	112622.0	1.3	5.9	<code>self.State.isEndFunc()</code>
187	84258	29997.0	0.4	1.6	<code>self.isEnd = self.State.isEnd</code>
188					
189					<code>#update Q values with max Q value for next state</code>
190	84258	100733.0	1.2	5.3	<code>self.new_Q[(i,j,action)] = round(mx_nxt_value,3)</code>
191					
192					<code>#copy new Q values to Q table</code>
193	94258	96267.0	1.0	5.0	<code>self.Q = self.new_Q.copy()</code>
194					<code>#print final Q table output</code>
195	1	144.0	144.0	0.0	<code>print(self.Q)</code>


```

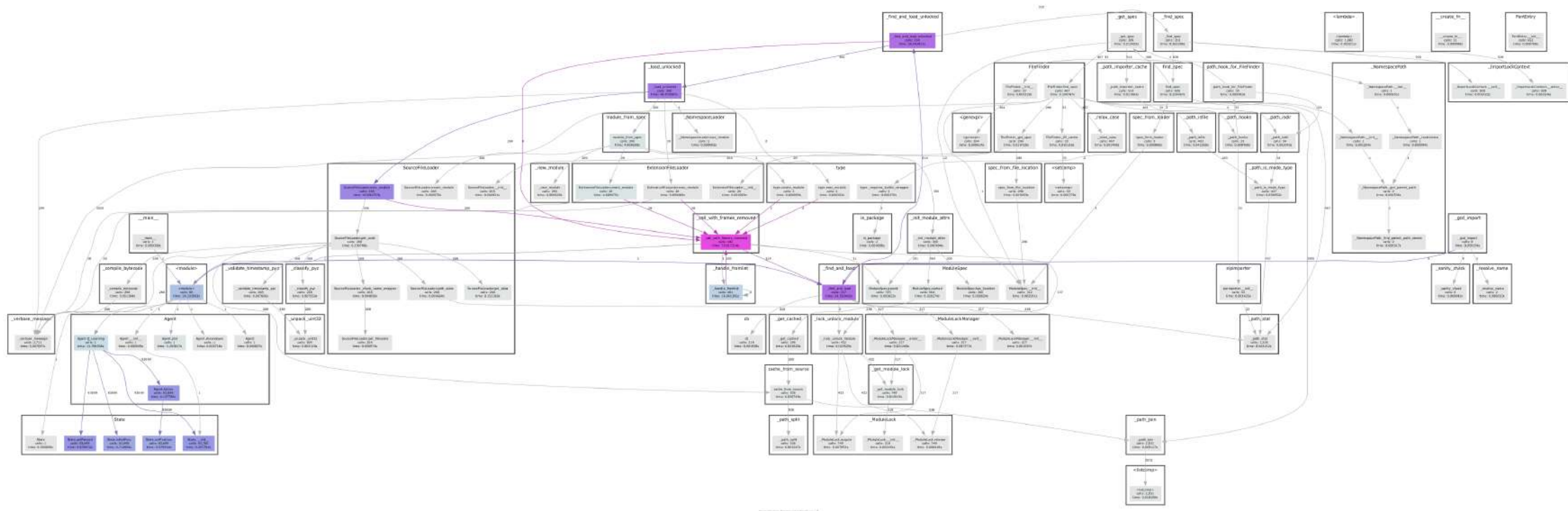
Total time: 0.828347 s
File: q_learning_profiling.py
Function: Action at line 111

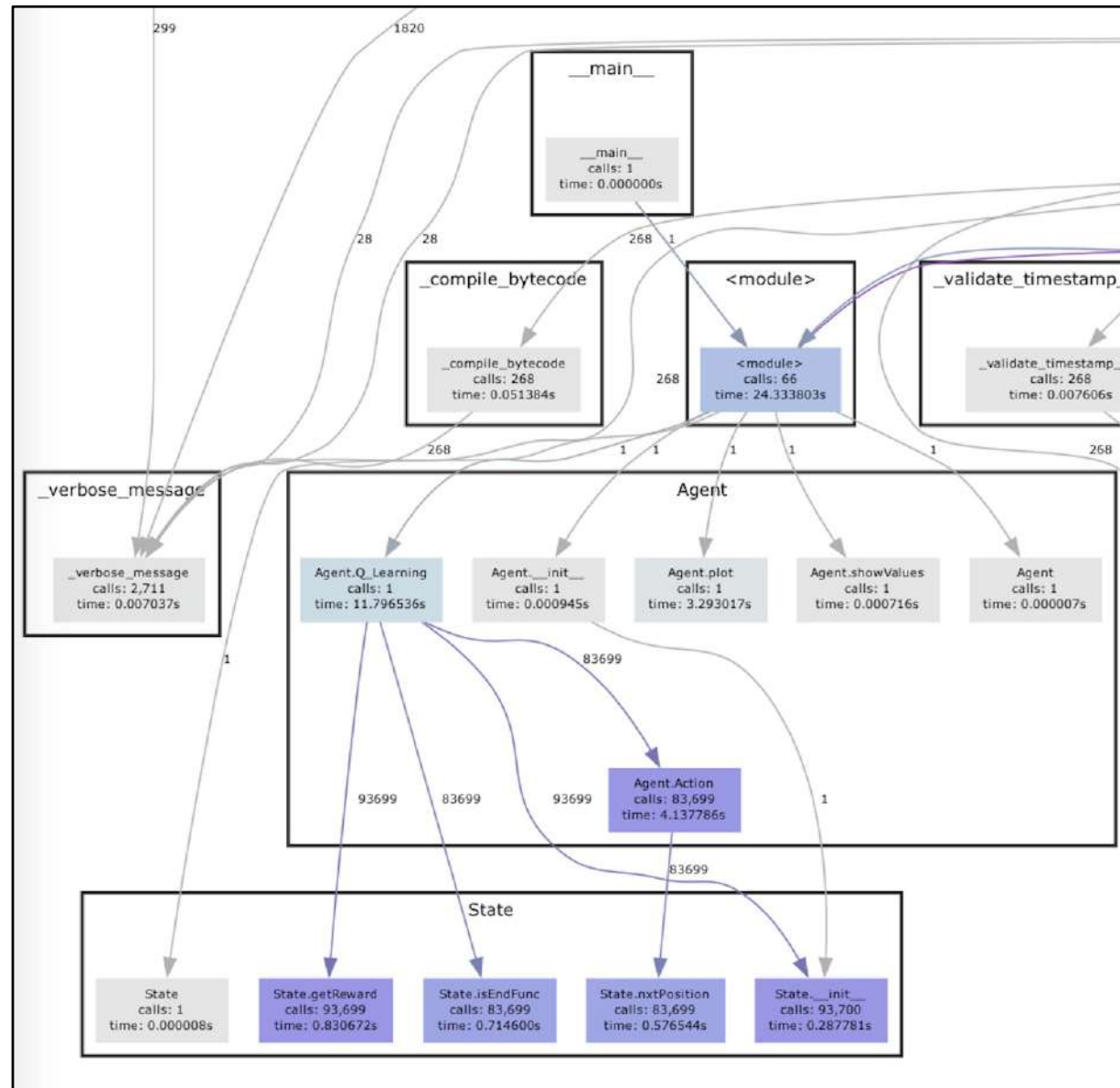
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
111					@profile
112					def Action(self):
113					#random value vs epsilon
114	83741	27544.0	0.3	3.3	rnd = random.random()
115					#set arbitraty low value to compare with Q values to find max
116	83741	19066.0	0.2	2.3	mx_nxt_reward = -10
117	83741	17424.0	0.2	2.1	action = None
118					
119					#9/10 find max Q value over actions
120	83741	24591.0	0.3	3.0	if(rnd > self.epsilon):
121					#iterate through actions, find Q value and choose best
122	376785	94958.0	0.3	11.5	for k in self.actions:
123					
124	301428	80642.0	0.3	9.7	i,j = self.State.state
125					
126	301428	108115.0	0.4	13.1	nxt_reward = self.Q[(i,j, k)]
127					
128	301428	73107.0	0.2	8.8	if nxt_reward >= mx_nxt_reward:
129	180726	37599.0	0.2	4.5	action = k
130	180726	35654.0	0.2	4.3	mx_nxt_reward = nxt_reward
131					
132					#else choose random action
133					else:
134	8384	170774.0	20.4	20.6	action = np.random.choice(self.actions)
135					
136					#select the next state based on action chosen
137	83741	118470.0	1.4	14.3	position = self.State.nxtPosition(action)
138	83741	20403.0	0.2	2.5	return position,action

Pycallgraph2: Dynamic Call Graph Generation

1. Requires pycallgraph2 and graphviz
2. `pycallgraph graphviz --output-file output.png q_learning_orig.py`





ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	3.438	3.438	3.743	3.743	~:0(<built-in method matplotlib.backends._macosx.show>)
46/44	0.7926	0.01801	0.7994	0.01817	~:0(<built-in method _imp.create_dynamic>)
1	0.4153	0.4153	0.9049	0.9049	q_learning_orig.py:141(Q_Learning)
83522	0.1256	1.504e-06	0.2928	3.506e-06	q_learning_orig.py:111(Action)
292	0.09859	0.0003376	0.09859	0.0003376	~:0(<method 'read' of '_io.BufferedReader' objects>)
1	0.07742	0.07742	0.1346	0.1346	backend_bases.py:2673(create_with_canvas)
7178	0.06007	8.369e-06	0.107	1.491e-05	inspect.py:626(cleandoc)
8403	0.05586	6.647e-06	0.1172	1.394e-05	~:0(<method 'choice' of 'numpy.random.mtrand.RandomState' objects>)
123564	0.055	4.451e-07	0.055	4.451e-07	~:0(<built-in method builtins.round>)
292	0.04718	0.0001616	0.04718	0.0001616	~:0(<built-in method marshal.loads>)
115	0.04713	0.0004098	0.04713	0.0004098	~:0(<method 'set_message' of 'matplotlib.backends._macosx.NavigationToolba
93522	0.04602	4.921e-07	0.04602	4.921e-07	q_learning_orig.py:33(getReward)
7	0.04531	0.006473	0.04541	0.006488	~:0(<method 'draw_path' of 'matplotlib.backends._backend_agg.RendererAgg'
83522	0.04079	4.884e-07	0.04079	4.884e-07	q_learning_orig.py:53(nxtPosition)

Presentations (1)

- **Goals**
 - Share your idea
 - Share your progress
 - Hone your presentation skills
- **Schedule**
 - Presentation duration: 1 minute (strict)
 - Schedule linked on Canvas
- **Instructions**
 - Create a single (= 1 = ONE) slide on which you answer the following questions:
 - What are you trying to do? I.e., what AI/ML algorithm/workload did you pick?)
 - How have others implemented and/or accelerated this algorithm?
 - What are you doing differently/better/etc.?
 - What have you accomplished so far?
 - What will you do next and what remains to be done until you can declare success?
- Save the slide in the shared Google folder. URL on Canvas
- A strong presentation earns 1 token.

Presentations (2)

Date	Week	Time	Name
Wed, April 22	4	4:55: PM	Eraganeni, Vamsidhar Reddy (Pronoun: He/Him)
Wed, April 22	4	4:57: PM	Kamarajugadda, Venkata Sriram (Pref: Venkata Sriram)
Wed, April 22	4	4:59: PM	Boss, David (Pronoun: He/Him)
Wed, April 22	4	5:01: PM	Hynes, Kay (Pref: Kay)
Wed, April 22	4	5:03: PM	Gilbert-Croysdale, Rebecca R. (Pronoun: She/Her)
Wed, April 22	4	5:05: PM	Charian, Sai Ganesh Reddy
Wed, April 29	5	4:55: PM	Vemula, Rajeshwar Reddy (Pronoun: he/him)
Wed, April 29	5	4:57: PM	Henrichs-Tarasenkov, Fyodor P. (Pref: Fedya, Pronoun: He/him)
Wed, April 29	5	4:59: PM	Herrera Moreno, Ivan (Pref: Ivan)
Wed, April 29	5	5:01: PM	Nguyen, Andrew (Pref: Bao)
Wed, April 29	5	5:03: PM	Chikani, Fatema Z. (Pronoun: She / Her)
Wed, April 29	5	5:05: PM	Munnangi, Pratibha (Pref: Pratibha, Pronoun: she/her)
Wed, May 6	6	4:55: PM	Ngo, Huy (Pref: Michael)
Wed, May 6	6	4:57: PM	Nguyen, Nhat M. (Pronoun: he/him)
Wed, May 6	6	4:59: PM	Naidu, Mahesh (Pref: Mahesh, Pronoun: He/Him)
Wed, May 6	6	5:01: PM	Rabiah, Yaqoub Y. (Pref: Yaqoub)
Wed, May 6	6	5:03: PM	Sabti, Saleh M. (Pref: Saleh, Pronoun: He/Him)
Wed, May 6	6	5:05: PM	Alali, Manaf A.
Wed, May 13	7	4:55: PM	Stanton, Andrew J. (Pref: Andrew, Pronoun: he/him)
Wed, May 13	7	4:57: PM	Bandla, Karthik
Wed, May 13	7	4:59: PM	Alrandi, Dawood A.
Wed, May 13	7	5:01: PM	Premarajan, Roshan B. (Pref: Roshan, Pronoun: hr/him)
Wed, May 13	7	5:03: PM	Vedantam, Venkata Krishna Kumar (Pronoun: His/Him)
Wed, May 13	7	5:05: PM	Esmaeil, Saleh A. (Pref: Sal, Pronoun: He/Him)

Wed, May 20	8	4:55: PM	Mills, Camden J. (Pref: Camden, Pronoun: He/Him)
Wed, May 20	8	4:57: PM	Day, Elliott F. (Pronoun: he/him)
Wed, May 20	8	4:59: PM	Mahajan, Ria (Pref: Ria)
Wed, May 20	8	5:01: PM	Khaddoori, Ibrahim A. (Pref: Ibrahim)
Wed, May 20	8	5:03: PM	AlQuzweni, Hussain A. (Pref: Hussain, Pronoun: he/him)
Wed, May 20	8	5:05: PM	Alotaibi, Mohammed M. (Pref: Mohammed)
Wed, May 27	9	4:55: PM	Wyse, Ian A. (Pronoun: he/him/his)
Wed, May 27	9	4:57: PM	Vanukuri, Sri Chandana Reddy
Wed, May 27	9	4:59: PM	Thakur, Amogh Vinod
Wed, May 27	9	5:01: PM	Kyatham, Vanisri (Pronoun: she/her)
Wed, May 27	9	5:03: PM	Vanamala, Naveen Babu (Pronoun: HE/HIM)
Wed, May 27	9	5:05: PM	Mallick, Devdutt S. (Pref: Devdutt, Pronoun: He/Him)
Wed, May 27	9	5:07: PM	Alshaiji, Mohammad A. (Pronoun: He/Him)
Wed, June 3	10	4:55: PM	Ramirez, Jose A. (Pronoun: he/him)
Wed, June 3	10	4:57: PM	Venkatesha Murthy, Veekshith (Pref: Veekshith, Pronoun: he, him)
Wed, June 3	10	4:59: PM	Handwerger, Adam L.
Wed, June 3	10	5:01: PM	Akula, Ranjith (Pronoun: he/him)
Wed, June 3	10	5:03: PM	Mickelson, Christian I. (Pref: Christian, Pronoun: He/Him)
Wed, June 3	10	5:05: PM	Shamszadeh, Navid (Pref: Navid, Pronoun: he/him)
Wed, June 3	10	5:07: PM	De Los Santos Razo, Jorge J. (Pronoun: He/Hims)